
HYPOLE: Hyperproperty-Guided Multi-Agent Reinforcement Learning under Partial Observation

Arshia Rafieioskouei¹ Tzu-Han Hsu¹ Matthew Lucas¹ Borzoo Bonakdarpour¹

Abstract

Formal specification is a powerful tool to guide the learning process and provides significant advantages over reward shaping: (1) mathematical rigor; (2) expressiveness to specify objectives and constraints, and (3) the ability to define tactics to achieve objectives. However, these benefits remain largely unexplored in the context of Multi-Agent Reinforcement Learning (MARL). This paper introduces HYPOLE, a novel framework for MARL under *partial observability*, where learning is guided by the expressive power of the so-called *hyperproperties* and, in particular, the temporal logic HyperLTL. We integrate Centralized Training for Decentralized Execution (CTDE) techniques with HYPOLE to synthesize *decentralized* policies, and our evaluation on SMAC, MessySMAC, and WildFire benchmark demonstrates clear advantages over baselines.

1. Introduction

Multi-Agent Reinforcement Learning (MARL) is challenging due to combinatorial scaling, and becomes even more demanding when agents must satisfy multiple interdependent objectives and constraints. Existing MARL techniques (Rashid et al., 2018; Sunehag et al., 2017; Son et al., 2019) are strangers to *symbolic* techniques that provide a wealth of powerful specification languages that express complex objectives and constraints among agents and guide the learning process. Moreover, shaped reward functions are often insufficient for expressing and guiding learning with respect to *relational* objectives among agents and their temporal behaviors (Alur et al., 2026). More importantly, a particular limitation of existing techniques is that shaped

rewards for a set of objectives are implicitly *universally quantified*; that is, the intended objective must hold for every pair of agents’ behaviors. This inherent universal strictness results in limiting the search space for better policies.

To explain the issue of universality, consider the following formula in predicate logic: $\varphi_1 = \forall x. \forall y. (x < y)$, where x and y range over integers. This formula is obviously false, but a weaker variation of this formula $\varphi_2 = \forall x. \exists y. (x < y)$ is valid. In fact, synthesizing a policy to select y in response to any choice of x is to learn a witnessing *Skolem function* (Skolem, 1879) such as $y = f(x) = x + 1$. In the world of MARL, virtually all existing approaches attempt to search for policies for objectives that are of the form $\forall \psi$, while many multi-agent requirements are of the form $\forall \exists \psi$, expressing that for every behavior of one agent, there exists a behavior of another agent, such that the objective is satisfied. For example, (Hsu et al., 2025; Beutner & Finkbeiner, 2024) formulate multi-agent planning problems, using $\forall \exists \psi$ specifications. Moreover, (Hsu et al., 2025) extends beyond multi-agent planning by encoding the single-agent planning problem DeepSea Treasure (Vamplew et al., 2011) as a multi-agent problem under a $\forall \exists \psi$ specification.

We propose a specification-guided method that boosts MARL under *partial observability*. We use the concept of *hyperproperties* and, in particular, the temporal logic HyperLTL (Clarkson et al., 2014) as our specification language to express the intended behavior of each agent as well as their *relational* (inter-dependent) objectives and constraints. Our proposed method HYPOLE (**H**yperproperties for **P**artially **O**bservable **L**earning **E**nvironments), is built on realistic multi-agent settings, assuming that agents operate in an environment modeled as a partially observable Markov decision process (POMDP) with the goal of learning a *decentralized* set of policies, one per agent (see Fig. 1).

Our Contributions. (1) We formulate the policy synthesis problem for a multi-agent system as a learning problem, where the goal is to maximize the probability of satisfying a HyperLTL specification that expresses a set of objectives and constraints under partial observability (Sec. 4). (2) We solve this problem by first transforming symbolic satisfaction of the HyperLTL specification for a POMDP into an optimization problem by defining continuous robustness

¹Department of Computer Science and Engineering, Michigan State University, MI, USA. Correspondence to: Arshia Rafieioskouei <rafieios@msu.edu>, Borzoo Bonakdarpour <borzoo@msu.edu>.

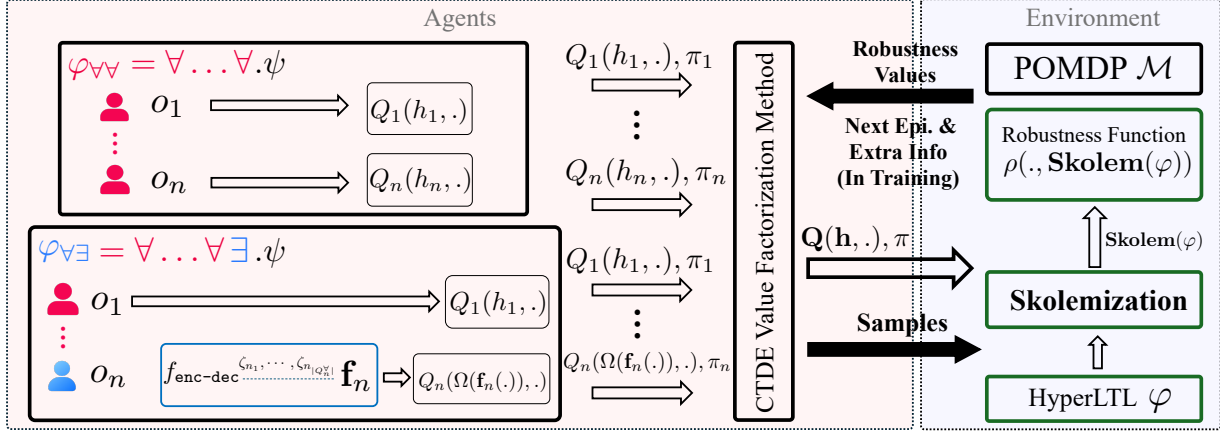


Figure 1. Overview of HYPOLE.

functions, then use off-the-shelf Centralized Training for Decentralized Execution (CTDE) algorithms to learn an optimal collection of decentralized policies (Sec. 5), and (3) we evaluate HYPOLE on scenarios from (i) the StarCraft II Multi-Agent Challenge (SMAC) (Samvelyan et al., 2019), (ii) MessySMAC (Phan et al., 2023), and (iii) the WildFire benchmark. Our results show that HYPOLE coupled with CTDE algorithms is more efficient and effective than the vanilla versions in handling complex requirements. We also show how HyperLTL assists in (1) faster learning of better policies, and (2) elegantly express various tactics (Sec. 6).

2. Related Work

We believe that the work closest to HYPOLE is HYPRL (Hsu et al., 2025), which studies hyperproperty-guided MARL in a fully observable and centralized setting. Both approaches leverage Skolemization, a well-known technique in logic for handling quantifier alternations. However, HYPOLE introduces a new challenge from the decentralized setting: the Skolem function does not have access to the traces of other agents when reasoning about its own behavior relative to theirs. Moreover, because HYPRL operates in a centralized setting, it does not scale well with increasing environment complexity or the number of agents. In contrast, HYPOLE addresses this scalability challenge by learning decentralized policies in partially observable multi-agent environments. Other work on specification-guided RL include (Jothimurugan et al., 2021; Li et al., 2017; Kuo et al., 2020; De Giacomo et al., 2019; Hasanbeig et al., 2019; Xu & Topcu, 2019; Yuan et al., 2019; Le et al., 2024). However, these approaches are either limited to single-agent tasks or fail to scale effectively in MARL settings due to the composition of MDPs. The extensions to multi-agent systems (Jothimurugan et al., 2022; ElSayed-Aly & Feng, 2022; Liu et al., 2024; León & Belardinelli, 2020; Hammond et al., 2021) either rely on centralized

constructions, employ enumerate-and-verify strategies, or support only co-safety properties. Several work integrate temporal logic constraints into POMDP planning, including finite-state controllers and barrier-based methods for LTL and DTL specifications (Sharan & Burdick, 2014; Ahmadi et al., 2020; Wang et al., 2021; Liu et al., 2021), but typically focus on single-agent or small settings. Furthermore, Inverse RL (Choi & Kim, 2011) recovers rewards from expert demonstrations, whereas HYPOLE uses only a hyperproperty specification and assumes no expert behavior.

3. Preliminaries

Let $\Delta(X) \triangleq \{p : X \mapsto [0, 1] \mid \sum_{x \in X} p(x) = 1\}$ be the probability simplex of a set X of random variables. We abbreviate tuples $\langle x_1, \dots, x_n \rangle$ as $\langle x_i \rangle_{i \in \{1, \dots, n\}}$.

3.1. Partially Observable Markov decision Processes

Definition 3.1. A *partially observable Markov decision process (POMDP)* is a tuple $\mathcal{M} = \langle S, I, A, P, R, \mathcal{O}, O, \text{AP}, L, \gamma \rangle$, where S is a finite set of *states*, I is *initial* distribution over S , i.e., $I \in \Delta(S)$, A is a finite *action* space, the transition function $P(s' \mid s, a)$ specifies the conditional probability of transitioning to $s' \in S$ after taking action $a \in A$ in $s \in S$. By taking a in s , the agent receives a scalar *reward* $R(s, a)$ and $\gamma \in (0, 1]$ is discount factor. An *observation* $o \in O$ is produced by observation probability function $\mathcal{O}(o \mid s, a)$ given the current s and a . AP denotes a finite set of *atomic propositions*, and $L : S \rightarrow 2^{\text{AP}}$ is a *labeling* function. $\square$

Example. Consider the environment in Fig. 2, a WildFire scenario on a 3×3 grid world with cells labeled $\{a, b, \dots, i\}$. A firefighter drone (FF ) is tasked to extinguish three fire zones , and a medical drone (Med ) whose objective is to rescue two victims . Agents can communicate only within a distance of three. Med is not

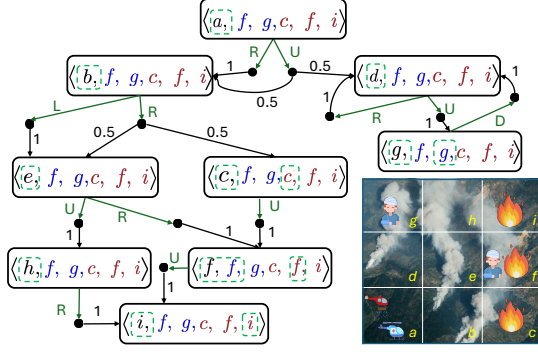


Figure 2. WildFire scenario and its POMDP.

allowed to enter fire-zone. The map is partially observable due to smoke and the agents can detect fire or victims only when they are in the same cell. The underlying state consists of the agents' positions and the locations of the two victims and three fire zones. Fig. 2 illustrates a POMDP, with state $\langle \text{pos}, \text{V1}, \text{V2}, \text{F1}, \text{F2}, \text{F3} \rangle$ (agent position, victim positions, and fire positions). Observations are partial; e.g., in state $\langle a, f, g, c, f, i \rangle$ the agent's observation is $\langle \bar{a} \rangle$.

A *history* is an observation-action sequence $h \triangleq o_0 \xrightarrow{a_0} o_1 \xrightarrow{a_1} \dots$, where for all $i \geq 0$, $o_i \in O$ and $a_i \in A$. A *sub-history* $h_{[\ell:k]}$ is a finite segment $o_\ell \xrightarrow{a_\ell} \dots \xrightarrow{a_{k-1}} o_k$, for $0 \leq \ell < k \leq |h|$. We denote the set of all finite histories by $\mathcal{H}^*$. A *policy* $\pi : \mathcal{H}^* \rightarrow A$ maps a history to a fixed action. A *path* in $\mathcal{M}$ is a sequence of states $\zeta \triangleq s_0 \xrightarrow{a_0} s_1 \xrightarrow{a_1} \dots$, where $s_i \in S$ and $a_i \in A$ for all $i \geq 0$. A *sub-path* $\zeta_{[\ell:k]}$ is a finite segment $s_\ell \xrightarrow{a_\ell} \dots \xrightarrow{a_{k-1}} s_k$, for $0 \leq \ell < k \leq |\zeta|$. Given a path ζ , the *trace* t of ζ is a sequence of labels $\text{Tr}(\zeta) \triangleq t(0)t(1)\dots$, such that $t(i) = L(s_i)$ for all $i \geq 0$. We write $\mathcal{Z}^*$ to denote the set of all finite paths, and $\text{Traces}(\mathcal{Z}^*)$ to denote the set of all finite traces of $\mathcal{M}$.

3.2. Finite Semantics for HyperLTL

HyperLTL syntax (Clarkson et al., 2014) is as follows:

$$\varphi ::= \exists \tau. \varphi \mid \forall \tau. \varphi \mid \psi \quad \psi ::= p_\tau \mid \neg \psi \mid \psi \vee \psi \mid \bigcirc \psi \mid \psi \mathcal{U} \psi,$$

where $p \in \text{AP}$ is an atomic proposition and τ is a trace variable. The second rule yields standard LTL formulas, with Boolean connectives $\neg$, $\vee$, and temporal operators $\bigcirc$ ('next') and $\mathcal{U}$ ('until'). Other Boolean and temporal operators are defined as syntactic sugar: $\text{true} \triangleq p_\tau \vee \neg p_\tau$, $\text{false} \triangleq \neg \text{true}$, $\psi_1 \rightarrow \psi_2 \triangleq \neg \psi_1 \vee \psi_2$, $\Diamond \psi \triangleq \text{true} \mathcal{U} \psi$, and $\Box \psi \triangleq \neg \Diamond \neg \psi$, where $\Diamond$ and $\Box$ denote the 'eventually' and 'always', respectively. In φ , $\exists \tau$ means "along some trace τ ", and $\forall \tau$ means "along all traces τ ". We write $\text{Vars}(\varphi)$ for the set of trace variables appearing in φ . A formula is *closed* if all $\tau \in \text{Vars}(\varphi)$ are quantified exactly once.

Since our setting is episodic and each episode terminates at a finite time, we adopt the finite-trace semantics of HyperLTL (Brett et al., 2017). I.e., formulas are interpreted

over finite trace assignments, given by a partial mapping $\Pi : \text{Vars}(\varphi) \rightarrow (2^{\text{AP}})^*$ that assigns each $\tau \in \text{Vars}(\varphi)$ to a finite trace. Given a trace assignment Π , a trace variable τ , and a finite trace $t \in (2^{\text{AP}})^*$, we write $\Pi[\tau \mapsto t]$ for the assignment that agrees with Π except that τ is mapped to t . We denote by $\Pi_\emptyset$ the empty trace assignment, and $t \in \Pi$ to refer to traces in the image of Π .

Given a closed HyperLTL formula $\varphi \triangleq Q_1 \tau_1 \dots Q_n \tau_n. \psi$, where each $Q_i \in \{\forall, \exists\}$, and ψ is an inner LTL formula, an *interpretation* is a tuple $\mathcal{T} \triangleq \langle T_{\tau_i} \rangle_{i \in \{1, \dots, |\text{Vars}(\varphi)|\}}$, where $T_{\tau_i} = \text{Traces}(\mathcal{Z}_i)$ is a set of traces that may be assigned to the corresponding trace variable τ_i . The satisfaction relation $\models$ maps a formula φ to a model $(\mathcal{T}, \Pi, i)$, where $i \in \mathbb{Z}_{\geq 0}$ indicates the current evaluation position. Formally:

$$\begin{aligned} (\mathcal{T}, \Pi, 0) &\models \exists \tau. \psi && \text{iff } \exists t \in T_\tau. (\mathcal{T}, \Pi[\tau \mapsto t], 0) \models \psi \\ (\mathcal{T}, \Pi, 0) &\models \forall \tau. \psi && \text{iff } \forall t \in T_\tau. (\mathcal{T}, \Pi[\tau \mapsto t], 0) \models \psi \\ (\mathcal{T}, \Pi, i) &\models p_\tau && \text{iff } p \in \Pi(\tau)(i) \\ (\mathcal{T}, \Pi, i) &\models \neg \psi && \text{iff } (\mathcal{T}, \Pi, i) \not\models \psi \\ (\mathcal{T}, \Pi, i) &\models \psi_1 \vee \psi_2 && \text{iff } (\mathcal{T}, \Pi, i) \models \psi_1 \text{ or } (\mathcal{T}, \Pi, i) \models \psi_2 \\ (\mathcal{T}, \Pi, i) &\models \bigcirc \psi && \text{iff } (\mathcal{T}, \Pi, i+1) \models \psi \\ &&& \text{and } \forall t \in \Pi. |t| > i+1 \\ (\mathcal{T}, \Pi, i) &\models \psi_1 \mathcal{U} \psi_2 && \text{iff } \exists j \geq i \text{ with } j < \min_{t \in \Pi} |t|, \\ &&& \text{s.t. } (\mathcal{T}, \Pi, j) \models \psi_2 \\ &&& \text{and } \forall k \in [i, j), (\mathcal{T}, \Pi, k) \models \psi_1 \end{aligned}$$

We say that an interpretation $\mathcal{T}$ satisfies a HyperLTL formula φ , written as $\mathcal{T} \models \varphi$, if $(\mathcal{T}, \Pi_\emptyset, 0) \models \varphi$.

Example. Consider the following arbitrary HyperLTL formula for the WildFire scenario:

$$\varphi_{\text{exp}} \triangleq \forall \tau_1 \exists \tau_2. \text{Dist}(\text{pos}_{\tau_1}, \text{pos}_{\tau_2}) < 3 \mathcal{U} (\text{pos}_{\tau_1} = i)$$

Here, FF corresponds to τ_1 and Med corresponds to τ_2 . The predicate $\text{Dist}(\text{pos}_{\tau_1}, \text{pos}_{\tau_2}) < 3$ enforces that they remain within distance 3, (until FF reaches cell i). Importantly, $\forall \exists$ captures inter-agent dependency by letting Med's decisions depend on FF's, whereas most specification-RL work focuses on $\forall \forall$, which cannot express this dependency.

4. Problem Statement

To bridge POMDP histories to HyperLTL interpretation, we define a function $\Phi : \mathcal{H}^* \times \mathcal{Z}^* \rightarrow [0, 1]$, which maps an action-observation history and a path to a scalar value in $[0, 1]$. We say that Φ is *path-consistent*, meaning that for any history h generated by a latent path ζ , $\zeta \in \arg \max_{\zeta' \in \mathcal{Z}^*} \Phi(h, \zeta')$. For a HyperLTL formula $\varphi \triangleq Q_1 \tau_1 \dots Q_n \tau_n. \psi$, where each $Q_i \in \{\forall, \exists\}$, let $\mathcal{D}_{\pi_i}$ denote the distribution over observation-action histories induced by a policy π_i and let $\mathcal{H}_i \sim \mathcal{D}_{\pi_i}$ denote a set of histories sampled from this distribution, for $1 \leq i \leq n$ and $\mathcal{H}_i \subseteq \mathcal{H}^*$. The policy π_i is associated with trace variable τ_i , for each $i \in \{1, \dots, |\text{Vars}(\varphi)|\}$, so that each trace variable ranges over the possible behaviors generated by its corresponding policy. The mapping Φ can be constructed by any technique such as belief-states constructions (Poupart & Boutilier, 2013; Rodriguez et al., 1999). For each trace variable $\tau_i \in \text{Vars}(\varphi)$, we introduce the set $\mathcal{Z}_i$ of paths,

Problem Statement

Given a POMDP $\mathcal{M}$ and a HyperLTL formula $\varphi = Q_1\tau_1 \dots Q_n\tau_n \cdot \psi$, our goal is to identify a tuple of n policies $\langle \pi_1^*, \dots, \pi_n^* \rangle$, such that:

$$\langle \pi_i^* \rangle_{i \in \{1, \dots, n\}} \in \left[\arg \max_{\langle \pi_i \rangle} \mathbb{P} \left[\langle \text{Traces} \left(\bigcup_{h \in (\mathcal{H}_i \sim \mathcal{D}_{\pi_i})} \{ \arg \max_{\zeta \in \mathcal{Z}^*} \Phi(h, \zeta) \} \rangle \rangle \models \varphi \right] \right]_{i \in \{1, \dots, n\}}$$

where $\mathcal{D}_{\pi_1}, \dots, \mathcal{D}_{\pi_n}$ are the distributions over a set of histories drawn by policies $\pi_1, \dots, \pi_n$, and $\bigcup_{h \in (\mathcal{H}_i \sim \mathcal{D}_{\pi_i})} \{ \arg \max_{\zeta \in \mathcal{Z}^*} \Phi(h, \zeta) \}$ is a set of paths $\mathcal{Z}_i$ associated with policy π_i .

Figure 3. Formal problem statement of HYPOLE.

where each path ζ in $\mathcal{Z}_i$ has the highest probability for each $h \in \mathcal{H}_i$ drawn from distribution $\mathcal{D}_{\pi_i}$ using policy π_i :

$$\mathcal{Z}_i \triangleq \bigcup_{h \in (\mathcal{H}_i \sim \mathcal{D}_{\pi_i})} \{ \arg \max_{\zeta \in \mathcal{Z}^*} \Phi(h, \zeta) \}$$

Then, the *family* of paths for each set $\mathcal{Z}_i$ of paths is defined as: $\mathbb{S} \triangleq \langle \mathcal{Z}_i \rangle_{i \in \{1, \dots, |\text{Vars}(\varphi)|\}}$. We write, $\mathcal{T} = \text{Traces}(\mathbb{S})$ as the tuple of sets of sampled traces. Also, we say a family of paths $\mathbb{S}$ (induced by each π_τ associated with each $\tau \in \text{Vars}(\varphi)$), satisfies a formula φ if $\langle \text{Traces}(\mathbb{S}) \rangle_{i \in \{1, \dots, |\text{Vars}(\varphi)|\}} \models \varphi$. The formal problem statement is shown in Fig. 3, where ‘ $\star$ ’ denotes optimality (e.g., $\pi^\star$ denotes an optimal policy). That is, the tuple of policies $\langle \pi_1^*, \dots, \pi_n^* \rangle$ maximizes the probability $\mathbb{P}$ such that the generated tuple of sets of traces from $\mathcal{M}$ satisfies φ .

Example. Consider the POMDP in Fig. 2 and φ_{exp} . Suppose FF samples histories with policy π_1 , $\mathcal{H}_{\tau_1} = \{h_1^1, h_1^2\}$:

$$\begin{aligned} h_1^1 &= \langle a \rangle \xrightarrow{R} \langle b \rangle \xrightarrow{R} \langle c, c \rangle \xrightarrow{U} \langle f, f, f \rangle \xrightarrow{U} \langle i, i \rangle \\ h_1^2 &= \langle a \rangle \xrightarrow{R} \langle b \rangle \xrightarrow{R} \langle e \rangle \xrightarrow{U} \langle h \rangle \xrightarrow{R} \langle i, i \rangle \end{aligned}$$

Likewise, agent Med, corresponding to τ_2 , samples histories using policy π_2 , $\mathcal{H}_{\tau_2} = \{h_2^1, h_2^2\}$:

$$\begin{aligned} h_2^1 &= \langle a \rangle \xrightarrow{U} \langle d \rangle \xrightarrow{U} \langle g, g \rangle \xrightarrow{D} \langle d \rangle \xrightarrow{R} \langle d \rangle \\ h_2^2 &= \langle a \rangle \xrightarrow{U} \langle b \rangle \xrightarrow{L} \langle e \rangle \xrightarrow{R} \langle f, f, f \rangle \xrightarrow{U} \langle i, i \rangle \end{aligned}$$

The set of paths associated with $\mathcal{H}_{\tau_i}$ is $\mathcal{Z}_{\tau_i} = \{ \arg \max_{\zeta \in \mathcal{Z}^*} \Phi(h_1^1, \zeta), \arg \max_{\zeta \in \mathcal{Z}^*} \Phi(h_1^2, \zeta) \}$, for $i \in \{1, 2\}$. We now compute the probability of satisfying φ_{exp} using $\mathcal{Z}_{\tau_1}$ and $\mathcal{Z}_{\tau_2}$ as follows, for $j \in \{1, 2\}$:

$$\text{Traces}(\langle \{ \arg \max_{\zeta \in \mathcal{Z}^*} \Phi(h_1^j, \zeta) \}, \mathcal{Z}_{\tau_2} \rangle) \models \varphi_{\text{exp}}$$

Here, $\arg \max_{\zeta \in \mathcal{Z}^*} \Phi(h_1^2, \zeta)$ serves as a witness for τ_2 in both satisfaction relations; hence the satisfaction probability of φ_{exp} under $\mathcal{H}_{\tau_1}$ and $\mathcal{H}_{\tau_2}$ is 1. However, if we replace the quantifiers in φ_{exp} with $\forall$, the satisfaction probability drops to 0.75 (check App. B for details).

5. Algorithmic Details of HYPOLE

Our solution to the problem in Fig. 3 has three steps. First, we apply Skolemization (Skolem, 1879) to eliminate quanti-

fier alternation (Sec. 5.1). Next, we define robustness functions to quantify and optimize satisfaction of the HyperLTL formula (Sec. 5.2). Finally, we lift the shared-environment POMDP to a Decentralized POMDP and apply CTDE algorithms to learn policies to solve the problem in Fig. 3.

5.1. Step 1: HyperLTL Skolemization

Let $\varphi = Q_1\tau_1 \dots Q_n\tau_n \cdot \psi(\tau_1, \dots, \tau_n)$ be a HyperLTL formula. Following (Hsu et al., 2025), we Skolemize φ to eliminate quantifier alternation. Let $Q^\exists = \{i \mid Q_i = \exists\}$ and $Q^\forall = \{j \mid Q_j = \forall\}$. For each $i \in Q^\exists$, let $Q_i^\forall = \{j \in Q^\forall \mid j < i\}$ denote the set of indices of all preceding universal quantifiers. For each $i \in Q^\exists$, a Skolem function $f_i : \mathcal{T}^{|Q_i^\forall|} \rightarrow \mathcal{T}$ is defined, reducing to a constant function when $Q_i^\forall = \emptyset$. A trace assignment Π is consistent with f_i if, for all $i \in Q^\exists$, $\Pi(\tau_j) \in \mathcal{T}$ for all $j \in Q_i^\forall$ and $\Pi(\tau_i) = f_i(\Pi(\tau_{i_1}), \Pi(\tau_{i_2}), \dots, \Pi(\tau_{i_{|Q_i^\forall|}}))$ for all $i \in Q^\exists$, where $Q_i^\forall = \{i_1 < \dots < i_{|Q_i^\forall|}\}$. If $(\mathcal{T}, \Pi, 0) \models \varphi$ holds for all trace assignments consistent with all f_i , then each f_i witnesses the satisfaction of φ (Winter & Zimmermann, 2025). For the inner LTL formula ψ , we obtain $\text{Skolem}(\psi)$ by substituting each proposition p_{τ_i} with p_{f_i} for all $p \in \text{AP}$ and $i \in Q^\exists$, thereby instantiating existential traces with their Skolem witnesses. The Skolemized formula is:

$$\text{Skolem}(\varphi) = \underbrace{\exists f_i(\tau_{i_1}, \dots, \tau_{i_{|Q_i^\forall|}})}_{\text{for each } i \in Q^\exists} \cdot \underbrace{\forall \tau_j}_{\text{for each } j \in Q^\forall} \cdot \text{Skolem}(\psi)$$

Based on this transformation, we rewrite the problem statement in Fig. 3 of Sec. 4. The updated statement is shown in Eq. (1), where $\text{Img}(f_i)$ denotes the set of traces obtained by applying the Skolem function to the preceding universally quantified traces (as detailed in App. C.1).

5.2. Step 2: Learning with Quantitative Semantics

We reformulate the HyperLTL satisfaction problem as an optimization problem under its quantitative semantics, where the Skolemized formula $\text{Skolem}(\varphi)$ is evaluated on tuples of histories $\langle h_1, \dots, h_n \rangle$ sampled from the POMDP $\mathcal{M}$. Let $\mathbb{R}$ be the set of real numbers, Ψ the set of all LTL formulas, and $f : 2^{\text{AP}} \rightarrow \mathbb{R}$ be a *valuation function* assigning real values to sets of atomic propositions. For a state $s \in S$ of $\mathcal{M}$, quantitative semantics are defined by predicates of the form $f(L(s)) < c$, where c is a user-specified threshold (see Fig. 9 in Appendix for full semantics). The robustness function $\rho : \text{Traces}(\mathcal{Z}^*) \times \Psi \rightarrow \mathbb{R}$ assigns a real-valued

$$\langle \pi_j^* \rangle_j \in \left[\arg \max_{\langle \pi_j \rangle} \mathbb{P} \left[\langle \text{Img}(\mathbf{f}_i) \rangle \bowtie \langle \text{Traces} \left(\bigcup_{h \in (\mathcal{H}_{i,j} \sim \mathcal{D}_{\pi_{i,j}})} \{ \arg \max_{\zeta \in \mathcal{Z}^*} \Phi(h, \zeta) \} \rangle \rangle \models \text{Skolem}(\varphi) \right] \right]_{i \in \mathbb{Q}^\exists, j \in \mathbb{Q}^\forall} \quad (1)$$

$$\langle \pi_\ell^* \rangle_\ell \in \left[\arg \max_{\langle \pi_\ell \rangle} \mathbb{P} \left[\rho(\text{zip}(\text{Tr}((\arg \max_{\zeta \in \mathcal{Z}^*} \Phi(h_\ell \sim \mathcal{D}_{\pi_\ell}, \zeta))_{[0:k_\ell]})), \psi) \xrightarrow{*} \rho_{\max} \right] \right]_{\ell \in \{1, \dots, n\}} \quad (2)$$

$$\langle \pi_i^* \rangle \sqcup \langle \pi_j^* \rangle \in \left[\arg \max_{\langle \pi_i \rangle \sqcup \langle \pi_j \rangle} \mathbb{P} \left[\rho \left(\text{zip} \left(\langle \text{Tr}((\arg \max_{\zeta \in \mathcal{Z}^*} \Phi(h_i \sim \mathcal{D}_{\pi_i}, \zeta))_{[0:k_i]}) \rangle \sqcup \langle \text{Tr}((\arg \max_{\zeta \in \mathcal{Z}^*} \Phi(h_j \sim \mathcal{D}_{\pi_j}, \zeta))_{[0:k_j]}) \rangle \right), \text{Skolem}(\psi) \right) \xrightarrow{*} \rho_{\max} \right] \right]_{i \in \mathbb{Q}^\exists, j \in \mathbb{Q}^\forall} \quad (3)$$

score to a finite trace and an LTL formula, bounded by $\rho_{\max}$ and $\rho_{\min}$. Formally, given an LTL formula ψ and a POMDP $\mathcal{M}$, we formulate the policy synthesis problem as:

$$\pi^* \in \arg \max_{\pi} \mathbb{P} \left[\rho(\text{Tr}((\arg \max_{\zeta \in \mathcal{Z}^*} \Phi(h, \zeta))_{[0:k]}), \psi) \xrightarrow{*} \rho_{\max} \right]$$

where $\xrightarrow{*}$ means convergence. I.e., π^* maximizes the probability that paths induced by histories under π satisfy ψ . Next, to compute robustness over multiple (universally quantified) traces, we define a zip operator as follows:

$$\text{zip}(\langle t_\ell \rangle_{\ell \in \{1, \dots, n\}}) \triangleq \langle t_\ell(0) \rangle_{\ell \in \{1, \dots, n\}} \cdots \langle t_\ell(k) \rangle_{\ell \in \{1, \dots, n\}}$$

where $k \triangleq \min_{\ell \in \{1, \dots, n\}} m_\ell$. E.g., given $t_1 = a_1 a_2 a_3$ and $t_2 = b_1 b_2 b_3$, their zipped trace is $\text{zip}(\langle t_1, t_2 \rangle) = \langle a_1, b_1 \rangle \langle a_2, b_2 \rangle \langle a_3, b_3 \rangle$. Thus, the optimization problem of computing a tuple of policies $\langle \pi_1^*, \dots, \pi_n^* \rangle$ that maximizes robustness can be stated as Eq. (2).

Now, let $\langle t_i \rangle_{i \in I}$ and $\langle t_j \rangle_{j \in J}$ be two trace tuples, where $I \cup J = \{1, \dots, n\}$ and $I \cap J = \emptyset$. To preserve index order when combining them, we use $\sqcup$, which produces a single tuple ordered by path indices. Based on this construction, satisfaction of the inner LTL body ψ is defined over tuples of histories. For each $i \in \mathbb{Q}^\exists$ and $j \in \mathbb{Q}^\forall$, a tuple $\langle h_1, \dots, h_n \rangle$ satisfies ψ if and only if, after combining the paths associated with histories via ordered union and zipping them into a joint trace, the resulting robustness score converges to $\rho_{\max}$ (details in App. C.2). Accordingly, we formulate the optimization problem in (3).

Observe Eq. (3), the robustness value ρ for an $\exists$ -quantified trace instantiated by a Skolem function depends on whether the trace induced by $h_i \sim \mathcal{D}_{\pi_i}$ can serve as a valid witness for the preceding $\forall$ -quantified traces. Concretely, the robustness of a Skolem witness $\mathbf{f}_i$ with respect to ψ is defined as $\rho(\mathbf{f}_i, \psi) \triangleq \rho(\mathbf{f}_i(\text{Tr}(\zeta_{i_1}), \dots, \text{Tr}(\zeta_{i_{|\mathbb{Q}^\forall|}})), \psi)$. As a result, the optimization of $\langle \pi_i^* \rangle$ is inherently coupled with $\langle \pi_j^* \rangle$, which is crucial for correctly capturing the semantics of HyperLTL formulas with quantifier alternation.

Theorem 5.1

Given a POMDP $\mathcal{M}$ and a HyperLTL formula φ , if Φ is path-consistent, then any tuple of policies $\langle \pi_i^* \rangle_{i \in \mathbb{Q}^\exists} \sqcup \langle \pi_j^* \rangle_{j \in \mathbb{Q}^\forall}$ that optimizes the Skolemized formula $\text{Skolem}(\varphi)$ also optimizes the probability of satisfying φ in $\mathcal{M}$ as defined in Fig. 3.

5.3. Step 3: MARL for HyperLTL

Now, we aim to synthesize policies for the universally quantified path variables and to learn Skolem functions that witness the existential quantifiers in the Skolemized HyperLTL formula. To this end, we adopt value-based CTDE paradigm, i.e., VDN (Suneag et al., 2017), QMIX (Rashid et al., 2018), and QTRAN (Son et al., 2019). In CTDE, training leverages global information, such as other agents' observations or the global state, while execution is decentralized and each agent acts based on its local observation.

Our objective is to compute tuples of optimal policies $\langle \pi_i^* \rangle_{i \in \mathbb{Q}^\exists}$ and $\langle \pi_j^* \rangle_{j \in \mathbb{Q}^\forall}$ that solve (3). For each $j \in \mathbb{Q}^\forall$, we construct an optimal policy $\pi_j^*(h_{j[0:k]})$ that depends solely on h_j . For each $i \in \mathbb{Q}^\exists$, the corresponding policy takes a history induced by the associated Skolem witness as an input. To formalize this, we introduce a trace-to-history consistency map $\mathfrak{H} : \mathcal{T} \rightarrow 2^{\mathcal{H}}$, where $\mathfrak{H}(t) \triangleq \{h \in \mathcal{H} \mid \text{Tr}(\arg \max_{\zeta \in \mathcal{Z}^*} \Phi(h, \zeta)) = t\}$. That is, $\mathfrak{H}(t)$ denotes the set of histories whose most likely latent path under Φ induces trace t . We assume that $\mathfrak{H}(t) \neq \emptyset$ for every $t \in \mathcal{T}$ under consideration. Since policies require a single history as input, we fix a function Sel , that maps a nonempty set X to an arbitrary member of X , and introduce $\Omega : \mathcal{T} \rightarrow \mathcal{H}$, and defined as, $\Omega(t) \triangleq \text{Sel}(\mathfrak{H}(t))$. Finally, for each $i \in \mathbb{Q}^\exists$, we construct an optimal policy that takes a history induced by the corresponding Skolem witness as an input, namely, $\pi_i^*(\Omega(\mathbf{f}_i(\text{Tr}(\zeta_{i_1[0:k]}), \dots, \text{Tr}(\zeta_{i_{|\mathbb{Q}^\forall|}[0:k]}))))$.

This construction implies that the decisions of the optimal policies associated with existential quantifiers depend on the optimal policies of the preceding universal quantifiers for capturing the inter-agent dependencies.

From POMDP to Dec-POMDP. Value-based CTDE algorithms are defined over Decentralized POMDPs (Dec-POMDP) (Oliehoek et al., 2016), represented as a tuple $G = \langle \mathcal{S}, \mathcal{I}, A, \mathcal{P}, \mathcal{R}, \mathcal{O}, \mathcal{D}, n, \text{AP}, L, \gamma \rangle$. It is important to note that a POMDP is a special case of a Dec-POMDP (Lauri et al., 2020), and our framework simultaneously samples all the histories associated with each quantifier from the POMDP. To make the shared POMDP compatible with CTDE algorithms, we formally lift the shared POMDP $\mathcal{M}$ for n agents into a Dec-POMDP. In this transformation, the state space remains the same, i.e., $\mathcal{S} \triangleq \mathcal{S}$ and $\mathcal{I} \in \Delta(\mathcal{S})$. The joint action space is defined as

the Cartesian product of the individual agent action spaces, i.e., $\mathcal{A} \triangleq A^n$ (we assume the same action space for all agents), and a joint action is given by $\mathbf{a} \triangleq \langle a_\ell \rangle_{\ell \in \{1, \dots, n\}}$. The transition function $\mathcal{P}(s' | s, \mathbf{a})$ specifies the conditional probability of transitioning to state $s' \in \mathcal{S}$ after taking joint action $\mathbf{a}$ in $s \in \mathcal{S}$. When taking $\mathbf{a}$ in s , the agents receive a scalar reward $\mathcal{R}(s_k, \mathbf{a}_k)$. The components γ , AP, and L are defined as in the POMDP case. The observation space in G is defined as $\mathcal{O} \triangleq O^n$, where each agent ℓ receives an individual observation from $o_\ell \in O$ based on the observation probability function $\mathcal{O}(o_\ell | s, a_\ell)$. The joint history is denoted by $\mathbf{h} \triangleq \langle h_\ell \rangle_{\ell \in \{1, \dots, n\}}$, where h_ℓ is the local history of agent $\ell \in \{1, \dots, n\}$. Each h_ℓ is obtained using $\mathcal{O}$.

Robustness Values as Reward Signals. Let the state of a zipped path at position k as $s_k = \text{zip}_k(\langle t_\ell \rangle_{\ell \in \{1, \dots, n\}})$. The immediate reward signal at step k is based on the scalar robustness value of the zipped trace from Eq. (3) defined as:

$$\mathcal{R}(s_k, \mathbf{a}_k) \triangleq \left[\rho \left(\text{zip}(\langle \text{Tr}(\arg \max_{\zeta \in \mathcal{Z}^*} \Phi(h_i, \zeta))_{[0:k+1]} \rangle) \sqcup \langle \text{Tr}(\langle \arg \max_{\zeta \in \mathcal{Z}^*} \Phi(h_j, \zeta) \rangle_{[0:k+1]} \rangle), \text{Skolem}(\psi) \right) \right]_{i \in \mathbb{Q}^\exists, j \in \mathbb{Q}^\forall}$$

Since paths are defined over state–action sequences, $\mathbf{a}_k$ is included in the zipped paths prefix to evaluate ρ . Moreover, because Dec-POMDP rewards are defined over the underlying state and joint action, we are allowed to assume that Φ is *path-consistent* to compute robustness-based reward and feedback from observation histories. This assumption is used only for reward computation during training, and is not needed to generate the optimal policies.

MARL Optimization Problem. The formal optimization problem is to find a joint policy $\pi(\mathbf{h}) \triangleq \langle \pi_\ell(h_\ell) \rangle_{\ell \in \{1, \dots, n\}}$ that maximizes the joint value function, defined as, $\mathcal{V}^\pi(\mathbf{h}_{[0:k]}) = \mathbb{E}[\mathcal{R}(s_k, \mathbf{a}_k) + \gamma \mathcal{V}^\pi(\mathbf{h}_{[0:k+1]})]$, where $\mathbf{a}_k = \pi(\mathbf{h}_{[0:k]})$.

As an alternative, many RL methods work with the joint history–action value function (Q-function) $\mathbf{Q}^\pi(\mathbf{h}, \mathbf{a})$, which evaluates taking joint action $\mathbf{a}$ at joint history $\mathbf{h}$ and continuing according to π , $\mathbf{Q}^\pi(\mathbf{h}_{[0:k]}, \mathbf{a}_k) = \mathbb{E}[\mathcal{R}(s_k, \mathbf{a}_k) + \gamma \mathbb{E}[\mathbf{Q}^\pi(\mathbf{h}_{[0:k+1]}, \pi(\mathbf{h}_{[0:k+1]})) | \mathbf{a}_k]]$.

CTDE methods learn per-agent utilities that are combined (e.g., via a mixing network) into a joint Q-function $\mathbf{Q}$ during centralized training. At execution time, agents act greedily using only local utilities, enabling decentralized control. Formally, CTDE aims to approximate $\mathbf{Q}^*(\mathbf{h}, \mathbf{a}) = \max_{\pi} \mathbf{Q}^\pi(\mathbf{h}, \mathbf{a})$ and induce the joint optimal policy $\pi^*(\mathbf{h}) \in \arg \max_{\mathbf{a}} \mathbf{Q}^*(\mathbf{h}, \mathbf{a})$.

Constructing Policies Using CTDE. From the given joint policy $\pi \triangleq \langle \pi_\ell \rangle_{\ell \in \{1, \dots, n\}}$, we construct two tuples of policies to the corresponding indices: $\langle \pi_i \rangle_{i \in \mathbb{Q}^\exists}$ and $\langle \pi_j \rangle_{j \in \mathbb{Q}^\forall}$.

In CTDE algorithms, each agent (corresponding to a trace variable in our setting) selects actions during execution

based solely on its own local observation history. Thus, for the policies associated with universally quantified traces, denoted by $\langle \pi_j \rangle_{j \in \mathbb{Q}^\forall}$, where each policy takes as input its local history $h_{j[0:k]}$, we can directly apply CTDE methods. In particular, we use robustness values as reward signals to approximate $\mathbf{Q}$ during centralized training, enabling the iterative construction of the policies $\langle \pi_j \rangle_{j \in \mathbb{Q}^\forall}$.

For policies associated with existentially quantified traces, denoted by $\langle \pi_i \rangle_{i \in \mathbb{Q}^\exists}$, each agent’s policy requires as input the history induced by the corresponding Skolem witness, which is produced by the Skolem function $\mathbf{f}_i$. To compute $\mathbf{f}_i$, access to the paths associated with the preceding universally quantified trace variables, namely $\zeta_{i_1[0:k]}, \dots, \zeta_{i_{|\mathbb{Q}^\forall|}[0:k]}$, is required. Such information is available during the centralized training, but not during decentralized execution; hence, during training, we construct a replay buffer $\mathcal{D}$ that stores sequences of the form $(o_i, s_{i_1}, \dots, s_{i_{|\mathbb{Q}^\forall|}})$ for each $i \in \mathbb{Q}^\exists$.

Next, we train $f_{\text{enc-dec}} : \mathcal{H} \rightarrow \mathcal{Z}^{|\mathbb{Q}^\forall|}$ model. During centralized training, we sample sequences from the replay buffer $\mathcal{D}$ and use them to train $f_{\text{enc-dec}}$ to predict the paths $(\hat{\zeta}_{i_1[0:k]}, \dots, \hat{\zeta}_{i_{|\mathbb{Q}^\forall|}[0:k]})$ given $h_{i[0:k]}$. The predicted paths serve as inputs to the Skolem function $\mathbf{f}_i$, enabling the construction of policies for existentially quantified traces; in particular, for each $i \in \mathbb{Q}^\exists$, we have, $\pi_i(\Omega(\mathbf{f}_i(\text{Tr}(\hat{\zeta}_{i_1[0:k]}), \dots, \text{Tr}(\hat{\zeta}_{i_{|\mathbb{Q}^\forall|}[0:k]}))))$.

Up to now, we have described how to learn $\langle \pi_i \rangle_{i \in \mathbb{Q}^\exists}$ and $\langle \pi_j \rangle_{j \in \mathbb{Q}^\forall}$ using CTDE. However, it is important that CTDE algorithms are designed to approximate a joint action–value function and to induce an optimal joint policy π^* . This objective does not, in general, imply that the resulting $\langle \pi_i^* \rangle_{i \in \mathbb{Q}^\exists} \sqcup \langle \pi_j^* \rangle_{j \in \mathbb{Q}^\forall}$ is optimal in all CTDE methods. In (Son et al., 2019), a formal property called *Individual–Global–Max* (IGM) is introduced. IGM property states that maximizing the joint Q-function is equivalent to independently maximizing each agent’s local function.

Definition 5.2. For a joint action–value function $\mathbf{Q}(\mathbf{h}, \mathbf{a})$, if there exist a tuple of functions $\langle Q_\ell \rangle_{\ell \in \{1, \dots, n\}}$ such that:

$$\arg \max_{\mathbf{a}} \mathbf{Q}(\mathbf{h}, \mathbf{a}) = \langle \arg \max_{a_\ell} Q_\ell(h_\ell, a_\ell) \rangle_{\ell \in \{1, \dots, n\}}$$

where $\langle Q_\ell \rangle_{\ell \in \{1, \dots, n\}}$ satisfies *IGM* for $\mathbf{Q}$ given $\mathbf{h}$. Among the CTDE algorithms considered, QTRAN satisfies IGM more generally, while QMIX satisfies it via monotonicity.

Theorem 5.3

Let $\mathcal{M}$ be a POMDP and φ a HyperLTL formula. If a CTDE algorithm learns an optimal joint action–value function $\mathbf{Q}^*$ satisfying the IGM property, then $\langle \pi_i^* \rangle_{i \in \mathbb{Q}^\exists}$ and $\langle \pi_j^* \rangle_{j \in \mathbb{Q}^\forall}$ induced by the joint policy π^* will optimize the satisfaction probability of $\text{Skolem}(\varphi)$.

Theorem 5.3 establishes the premise of Theorem 5.1, which in turn solves the original problem Fig. 3 (proofs in App. A).



Figure 4. HYPOLE + QMIX vs. QMIX on SMAC corridor map.

6. Experiments and Results

Implementation. HYPOLE is implemented on top of PyMARL repository (Samvelyan et al., 2019). We start with a Skolemized HyperLTL formula. Using the quantitative semantics described in Sec. 5.2, we construct robustness functions and use the resulting robustness values as reward signals in a cooperative MARL environment. Our objective is to synthesize policies π_ℓ for each agent $\ell \in \{1, \dots, n\}$, depending on the quantifier associated with its trace variable. For universally quantified traces, i.e., policies $\langle \pi_j \rangle_{j \in Q^\forall}$, each policy receives only the agent’s local history. In contrast, for existentially quantified traces, i.e., policies $\langle \pi_i \rangle_{i \in Q^\exists}$, the policy input is the history induced by the output of the corresponding Skolem function f_i .

We train an encoder-decoder model to predict the traces of the preceding universally quantified variables. During centralized training, mini-batches sampled from the replay buffer $\mathcal{D}$ are used to train the encoder-decoder model every β steps. The encoder-decoder model follows a feedforward autoencoder-style architecture. We then employ value-based CTDE algorithms, namely VDN, QMIX, and QTRAN to construct the policies. In addition, we evaluate Independent Q-Learning (IQL) (Tan, 1993). For IQL, we consider only $\forall\forall$ -form HyperLTL specifications, which evaluates using local observations alone. Additional details on the experimental setting are provided in App. D.

Case Studies. We conduct our experiments on scenarios from the StarCraft Multi-Agent Challenge (SMAC) (Samvelyan et al., 2019), which is based on the strategy game StarCraft II and focuses on cooperative micromanagement tasks. In addition, we use MessySMAC (Phan et al., 2023), which introduces observation stochasticity ϕ and randomized initialization with K random steps before each episode starts. This case study adds significant challenges for finding optimal policies in SMAC. Finally, we extend the WildFire benchmark introduced in (Hsu et al., 2025) to a partially observable setting. More details on SMAC, MessySMAC, and WildFire are provided in Apps. E and F.

Experimental Organization. We evaluate HYPOLE on nine SMAC scenarios, including both hard and super-hard scenarios (see Table 1 in the Appendix), five MessySMAC scenarios, and four WildFire scenarios (see Table 2 in the Appendix). We (1) assess the effect of using robustness val-

ues from $\forall\forall$ -HyperLTL specifications as reward signals in MARL, compared to baseline MARL algorithms using standard shaped rewards; (2) compare $\forall\exists$ -HyperLTL specifications with $\forall\forall$ -HyperLTL specifications in terms of learning efficiency and performance; (3) investigate how HyperLTL enables expressing distinct combat tactics, and (4) discuss HyperLTL specifications that fail to provide effective guidance for learning. All specifications used in our experiments are presented in App. G.

HYPOLE vs. Shaped Rewards. We compare the robustness values generated by HYPOLE using $\forall\forall$ -HyperLTL specifications with the shaped reward functions introduced in (Samvelyan et al., 2019). Fig. 5 reports the median test win rate in SMAC (i.e., elimination of all enemy agents) of HYPOLE + MARL, where $\text{MARL} \in \{\text{VDN}, \text{QMIX}, \text{QTRAN}, \text{IQL}\}$, compared against vanilla MARL using shaped rewards. In Fig. 5a, we present our results for the $\varphi_{\text{focus-fire}}$ on the 3s5z map, where $\varphi_{\text{focus-fire}}$ encourages allied agents to focus fire on enemies. Across all settings for this map, HYPOLE + MARL outperforms the vanilla MARL baselines. Notably, HYPOLE + QMIX (orange line) initially underperforms vanilla QMIX (blue line), but after $\approx 700\text{K}$ environment steps the agents learn the focus fire tactic and subsequently surpass vanilla QMIX. Similarly, on the 8m map Fig. 5b, HYPOLE + MARL using $\varphi_{\text{focus-fire}}$ outperforms the vanilla MARL baselines in all cases. In particular, HYPOLE + IQL (gray line) not only surpasses vanilla IQL (pink line), but also outperforms vanilla QTRAN (purple line), despite QTRAN being a more complex CTDE method. Moreover, HYPOLE + IQL remains competitive with vanilla VDN (green line) and QMIX (blue line). We also evaluate $\varphi_{\text{focus-fire}}$ on bane_vs_bane map Fig. 5c, where HYPOLE + QMIX (orange line) and HYPOLE + VDN (red line) significantly outperform vanilla QMIX (blue line) and VDN (green line).

Fig. 5d evaluates HYPOLE + MARL using φ_{kite} formula on 5m_vs_6m, where φ_{kite} encourages agents to attack only when their weapon is ready. For all cases on this map, HYPOLE + MARL competes with the vanilla MARL baselines. In Fig. 5e, we evaluate the φ_{medivac} on MMM map, where φ_{medivac} encourages Medivac agents to move toward low-health allies to heal them. In all cases, HYPOLE + MARL outperforms the vanilla baselines. In particular, HYPOLE + QTRAN (brown line) and HYPOLE + IQL (gray line) substantially improve over vanilla QTRAN (purple line) and IQL (pink line), respectively. We find HYPOLE particularly promising on the corridor map (Fig. 5f), which is widely regarded as one of the hardest SMAC maps. We use $\varphi_{\text{corridor}}$ formula, which encourages agents to hold the choke point and damage enemies from that position. Using this formula, HYPOLE + QMIX (orange line) achieves a median win rate of up to $\approx 60\%$, substantially outperforming vanilla QMIX (blue line), which achieves a win rate of less than 5%. To

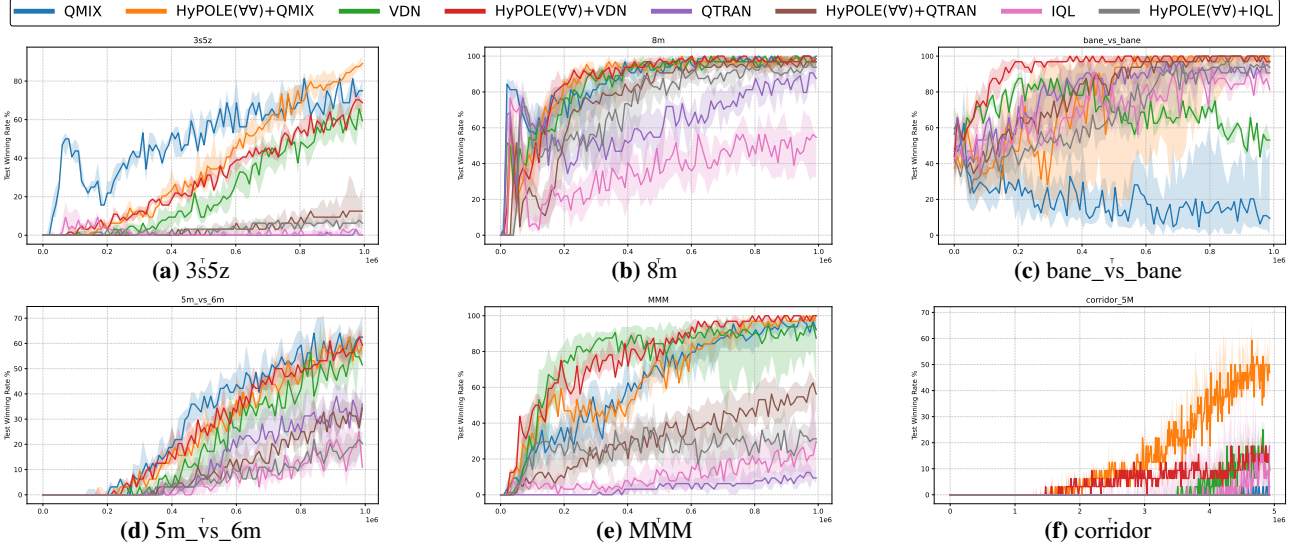


Figure 5. Learning curves on different SMAC maps, showing median test win rate with 25-75% percentile.

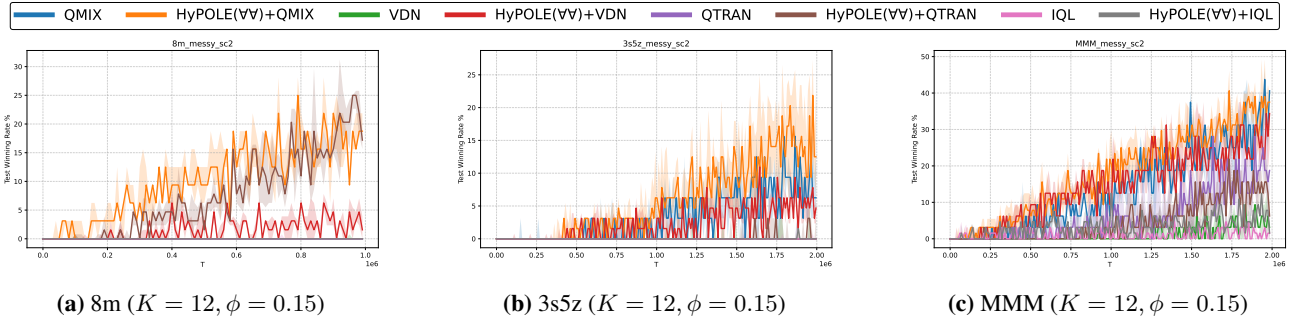


Figure 6. Learning curves on different MessySMAC maps, showing median test win rate with 25-75% percentile.

illustrate this, Fig. 4 shows snapshots of agent behavior after 5M training steps. Under HYPOLE +QMIX, agents form a defensive line at the choke point. In contrast, vanilla QMIX agents lose formation and split into three groups. One group of three agents, marked by $\checkmark$, is surrounded by five enemies and may still survive because Zealots are stronger than Zerglings. However, the single agent marked by $\triangle$ is surrounded by more than ten Zerglings and is likely to be defeated. This then exposes the two agents marked by $\triangle$, leading to the loss of three allies and eventually the remaining agents, who are heavily outnumbered.

We also compare HYPOLE +MARL against vanilla MARL on MessySMAC, with results shown in Fig. 6. In 8m map Fig. 6a, we use the $\varphi_{\text{focus-fire}}$ formula. Across all cases, HYPOLE +CTDE outperforms vanilla CTDE. In particular, HYPOLE +QTRAN (brown line) and HYPOLE +QMIX (orange line) achieve win rates of up to 25%, whereas the vanilla baselines fail to achieve any wins. Similarly, we use $\varphi_{\text{focus-fire}}$ in 3s5z map Fig. 6b. In this scenario, HYPOLE +VDN (red line) outperforms vanilla VDN, which fails to

achieve any wins. Moreover, HYPOLE +QMIX (orange line) performs slightly better than vanilla QMIX (blue line). In MMM map Fig. 6c, we use the φ_{medivac} formula. Here, HYPOLE +VDN (red line) achieves a win rate of up to $\approx 35\%$, substantially outperforming vanilla VDN (green line), which remains below 10%. The remaining SMAC and MessySMAC maps, together with the full WildFire comparison against hand-built rewards, are provided in App. H.1.

$\forall\exists$ vs. $\forall\forall$ Specifications. We study the effect of using $\forall\exists$ variants of the φ_{medivac} and $\varphi_{\text{wildfire}}$ specifications, and show that they can lead to improved policies by expanding the policy search space. In the MMM scenario in SMAC and MessySMAC, there is an inter-agent dependency between the Medivac and the remaining agents. To capture this, we change the quantifier associated with the Medivac from $\forall$ to $\exists$ in φ_{medivac} . In MMM scenario in SMAC Fig. 7a, HYPOLE ($\forall\exists$)+QMIX (orange line) and HYPOLE ($\forall\exists$)+VDN (red line) outperform HYPOLE ($\forall\forall$)+QMIX (blue line) and HYPOLE ($\forall\forall$)+VDN (green line). For the MessySMAC benchmark, Fig. 6c shows that HYPOLE ($\forall\forall$)+QMIX has

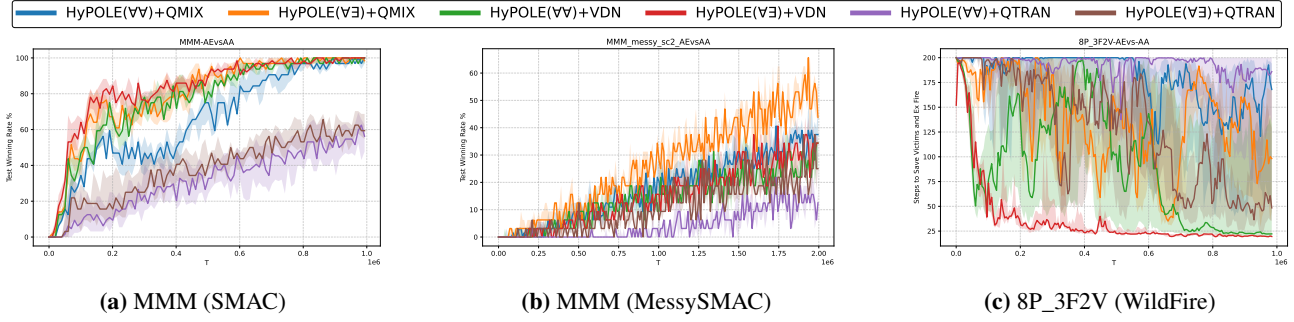


Figure 7. $\forall\forall$ vs. $\forall\exists$ learning curves across MessySMAC, SMAC, and WildFire, showing median test win rate with 25–75% percentiles.

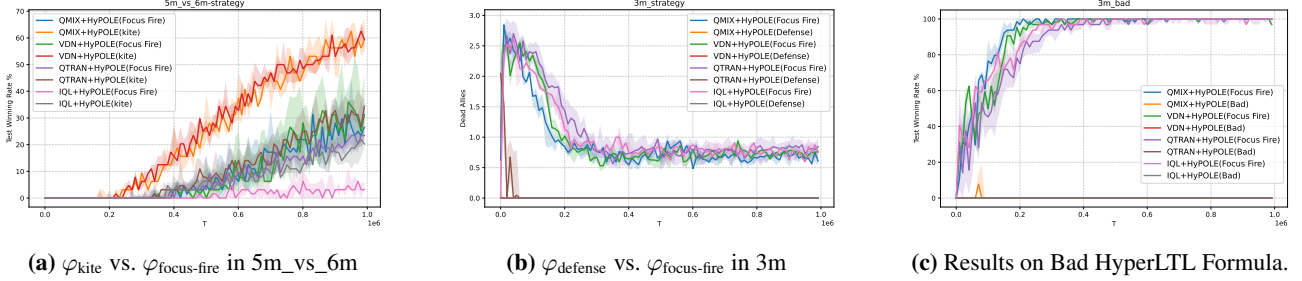


Figure 8. Expressing tactics using HYPOLE, and results on a bad HyperLTL formula.

only a slight edge over the vanilla baseline. In contrast, Fig. 7b shows that HYPOLE + QMIX($\forall\exists$) (orange line) performs better than HYPOLE + QMIX($\forall\forall$) (blue line) and, consequently, the vanilla QMIX baseline. Similarly, in Fig. 6c, HYPOLE + QTRAN($\forall\forall$) performs worse than the vanilla baseline. However, using HYPOLE + QTRAN($\forall\exists$) (brown line) in Fig. 7b improves the performance of HYPOLE. In Fig. 7c, we observe that the $\forall\exists$ version of the $\varphi_{wildfire}$ formula helps HYPOLE achieve better results in the WildFire 8P_3F2V scenario. Specifically, HYPOLE + VDN($\forall\exists$) (red line) performs substantially better than HYPOLE + VDN($\forall\forall$) (green line), converging after 500K training steps compared to 800K steps. Additional results comparing $\forall\exists$ and $\forall\forall$ are provided in App. H.2.

Tactics as HyperLTL Formulas. A key advantage of HYPOLE is learning policies that maximize satisfaction of HyperLTL-encoded tactics. In Fig. 8a, we study the 5m_vs_6m scenario and show that the kiting (φ_{kite}) yields consistently higher win rates than the focus firing ($\varphi_{focus-fire}$), and this trend is consistently evident in all cases. In 3m map Fig. 8b, we further demonstrate the expressiveness of HYPOLE, by specifying a defensive tactic using $\varphi_{defense}$, which encourages allied agents to retreat from enemies and results in zero ally casualties. This experiment shows that diverse tactics can be encoded in HyperLTL and compiled into policies via HYPOLE.

Quality of HyperLTL Specifications. We also study the effect of weakening a HyperLTL specification by removing a subformula from the $\varphi_{focus-fire}$, resulting in the φ_{bad} . This modification eliminates the constraint that encourages

agents to shoot enemies. In Fig. 8c, HYPOLE with φ_{bad} formula achieves an almost zero win rate. This shows that the performance of HYPOLE strongly depends on the quality and completeness of the underlying HyperLTL specification.

7. Conclusion

We presented HYPOLE, a specification-guided MARL framework that enables model-free synthesis of decentralized policies from HyperLTL specifications over POMDPs. HYPOLE effectively handles complex multi-agent objectives under partial observability by leveraging the semantics of HyperLTL to guide the learning process by off-the-shelf MARL algorithms. HYPOLE is fully implemented and we showed superior performance over baselines, namely, vanilla versions of VDN, QMIX, QTRAN, and IQL, across a set of diverse case studies.

Limitations. Admittedly, the main challenge in using HYPOLE is writing precise HyperLTL specifications. Low-quality specifications may hinder the learning process. We believe this limitation opens up exciting future work on AI-enabled specification generation. Moreover, the current version of HYPOLE is limited to discrete-action settings because it relies on value-based CTDE methods, which we choose over actor-critic methods due to their IGM-like guarantees. Finally, HYPOLE is not the method of choice when the task does not involve relational dependencies or temporal behaviors between agents and can instead be well captured by standard reward design, such as tasks with independent objectives or simple non-temporal requirements.

Acknowledgment

This work is partially sponsored by the United States NSF Award SaTC 2245114.

Impact Statement

This work takes a step toward bringing expressive specifications, in the form of hyperproperties, into practical MARL for partially observable and uncontrolled environments. By allowing tactics to be stated as precise mathematical formulas, HYPOLE can support the design of interpretable and verifiable behaviors in autonomous domains such as drone swarms. At the same time, the use of expressive specifications introduces potential risks. Malicious or adversarial specifications could be used to induce harmful behaviors, and poorly designed specifications may lead to unintended consequences. These risks highlight the importance of careful specification design, validation, and human oversight when deploying specification-guided MARL systems.

References

- Ahmadi, M., Singletary, A., Burdick, J. W., and Ames, A. D. Barrier functions for multiagent-pomdps with dtl specifications. In *2020 59th IEEE Conference on Decision and Control (CDC)*, pp. 1380–1385. IEEE, 2020. URL <http://dx.doi.org/10.1109/CDC42340.2020.9304266>.
- Alur, R., Bansal, S., Bastani, O., and Jothimurugan, K. Specification-guided reinforcement learning. *Commun. ACM*, 69(2):80–87, January 2026. ISSN 0001-0782. doi: 10.1145/3744706. URL <https://doi.org/10.1145/3744706>.
- Beutner, R. and Finkbeiner, B. Non-deterministic planning for hyperproperty verification. *Proceedings of the International Conference on Automated Planning and Scheduling*, 34(1):25–30, May 2024. doi: 10.1609/icaps.v34i1.31457. URL <https://ojs.aaai.org/index.php/ICAPS/article/view/31457>.
- Brett, N., Siddique, U., and Bonakdarpour, B. Rewriting-based runtime verification for alternation-free HyperLTL. In *Proceedings of the 23rd International Conference on Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, pp. 77–93, 2017. URL http://dx.doi.org/10.1007/978-3-662-54580-5_5.
- Choi, J. and Kim, K.-E. Inverse reinforcement learning in partially observable environments. *Journal of Machine Learning Research*, 12(21):691–730, 2011. URL <http://jmlr.org/papers/v12/choi11a.html>.
- Clarkson, M. R., Finkbeiner, B., Koleini, M., Micinski, K. K., Rabe, M. N., and Sánchez, C. Temporal logics for hyperproperties. In *Proceedings of the 3rd Conference on Principles of Security and Trust POST*, pp. 265–284, 2014. URL http://dx.doi.org/10.1007/978-3-642-54792-8_15.
- De Giacomo, G., Iocchi, L., Favorito, M., and Patrizi, F. Foundations for restraining bolts: Reinforcement learning with ltlf/ldlf restraining specifications. *Proceedings of the International Conference on Automated Planning and Scheduling*, 29(1):128–136, Jul. 2019. doi: 10.1609/icaps.v29i1.3549. URL <https://ojs.aaai.org/index.php/ICAPS/article/view/3549>.
- ElSayed-Aly, I. and Feng, L. Logic-based reward shaping for multi-agent reinforcement learning, 2022. URL <https://arxiv.org/abs/2206.08881>.
- Hammond, L., Abate, A., Gutierrez, J., and Wooldridge, M. Multi-agent reinforcement learning with temporal logic specifications. In *Proceedings of the 20th International Conference on Autonomous Agents and MultiAgent Systems, AAMAS ’21*, pp. 583–592, Richland, SC, 2021. International Foundation for Autonomous Agents and Multiagent Systems. ISBN 9781450383073.
- Hasanbeig, M., Abate, A., and Kroening, D. Logically-constrained reinforcement learning, 2019. URL <https://arxiv.org/abs/1801.08099>.
- Hsu, T.-H., Rafieioukouei, A., and Bonakdarpour, B. Hyprl: Reinforcement learning of control policies for hyperproperties. In Belgrave, D., Zhang, C., Lin, H., Pascanu, R., Koniusz, P., Ghassemi, M., and Chen, N. (eds.), *Advances in Neural Information Processing Systems*, volume 38, pp. 52298–52329. Curran Associates, Inc., 2025. URL https://proceedings.neurips.cc/paper_files/paper/2025/file/4b58601df13de0fab64698b6d01f13c3-Paper-Conference.pdf.
- Jothimurugan, K., Bansal, S., Bastani, O., and Alur, R. Compositional reinforcement learning from logical specifications. In Ranzato, M., Beygelzimer, A., Dauphin, Y., Liang, P., and Vaughan, J. W. (eds.), *Advances in Neural Information Processing Systems*, volume 34, pp. 10026–10039. Curran Associates, Inc., 2021. URL https://proceedings.neurips.cc/paper_files/paper/2021/file/531db99cb00833bcd414459069dc7387-Paper.pdf.
- Jothimurugan, K., Bansal, S., Bastani, O., and Alur, R. Specification-guided learning of nash equilibria with high social welfare. In *International Conference on Computer Aided Verification*, pp. 343–363. Springer, 2022. URL http://dx.doi.org/10.1007/978-3-031-13188-2_17.
- Kuo, Y.-L., Katz, B., and Barbu, A. Encoding formulas as deep networks: Reinforcement learning for zero-shot execution of ltl formulas. In *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp.

- 5604–5610. IEEE, 2020. URL <http://dx.doi.org/10.1109/IROS45743.2020.9341325>.
- Lauri, M., Pajarinen, J., and Peters, J. Multi-agent active information gathering in discrete and continuous-state decentralized pomdps by policy graph improvement. *Autonomous Agents and Multi-Agent Systems*, 34 (2):42, 2020. doi: 10.1007/s10458-020-09467-6. URL <https://doi.org/10.1007/s10458-020-09467-6>.
- Le, X.-B., Wagner, D., Witzman, L., Rabinovich, A., and Ong, L. Reinforcement learning with ltl and omega-regular objectives via optimality-preserving translation to average rewards. In Globerson, A., Mackey, L., Belgrave, D., Fan, A., Paquet, U., Tomczak, J., and Zhang, C. (eds.), *Advances in Neural Information Processing Systems*, volume 37, pp. 117109–117132. Curran Associates, Inc., 2024. doi: 10.52202/079017-3718. URL https://proceedings.neurips.cc/paper_files/paper/2024/file/d4857f724cc1af4c8f1e18032426aa2e-Paper-Conference.pdf.
- León, B. G. and Belardinelli, F. Extended markov games to learn multiple tasks in multi-agent reinforcement learning, 2020. URL <https://arxiv.org/abs/2002.06000>.
- Li, X., Vasile, C.-I., and Belta, C. Reinforcement learning with temporal logic rewards. In *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 3834–3839. IEEE, 2017. URL <http://dx.doi.org/10.1109/IROS.2017.8206234>.
- Liu, C., Cong, J., Chen, B., Jin, Y., and Zhu, E. Guiding multi-agent multi-task reinforcement learning by a hierarchical framework with logical reward shaping, 2024. URL <https://arxiv.org/abs/2411.01184>.
- Liu, J., Rosen, E., Zheng, S., Tellex, S., and Konidaris, G. Leveraging temporal structure in safety-critical task specifications for pomdp planning. In *Proc. RSS Workshop Robot. People*, 2021.
- Oliehoek, F. A., Amato, C., et al. *A concise introduction to decentralized POMDPs*, volume 1. Springer, 2016.
- Phan, T., Ritz, F., Altmann, P., Zorn, M., Nüßlein, J., Kölle, M., Gabor, T., and Linnhoff-Popien, C. Attention-based recurrence for multi-agent reinforcement learning under stochastic partial observability. In *International Conference on Machine Learning*, pp. 27840–27853. PMLR, 2023.
- Poupart, P. and Boutilier, C. Value-directed belief state approximation for pomdps, 2013. URL <https://arxiv.org/abs/1301.3887>.
- Rashid, T., Samvelyan, M., Schroeder, C., Farquhar, G., Foerster, J., and Whiteson, S. QMIX: Monotonic value function factorisation for deep multi-agent reinforcement learning. In Dy, J. and Krause, A. (eds.), *Proceedings of the 35th International Conference on Machine Learning*, volume 80 of *Proceedings of Machine Learning Research*, pp. 4295–4304. PMLR, 10–15 Jul 2018. URL <https://proceedings.mlr.press/v80/rashid18a.html>.
- Rodriguez, A., Parr, R., and Koller, D. Reinforcement learning using approximate belief states. In Solla, S., Leen, T., and Müller, K. (eds.), *Advances in Neural Information Processing Systems*, volume 12. MIT Press, 1999. URL https://proceedings.neurips.cc/paper_files/paper/1999/file/158fc2ddd52ec2cf54d3c161f2dd6517-Paper.pdf.
- Samvelyan, M., Rashid, T., Schroeder de Witt, C., Farquhar, G., Nardelli, N., Rudner, T. G. J., Hung, C.-M., Torr, P. H. S., Foerster, J., and Whiteson, S. The starcraft multi-agent challenge. In *Proceedings of the 18th International Conference on Autonomous Agents and MultiAgent Systems, AAMAS ’19*, pp. 2186–2188, Richland, SC, 2019. International Foundation for Autonomous Agents and Multiagent Systems. ISBN 9781450363099.
- Sharan, R. and Burdick, J. Finite state control of pomdps with ltl specifications. In *2014 American Control Conference*, pp. 501–508. IEEE, 2014. URL <http://dx.doi.org/10.1109/ACC.2014.6858909>.
- Skolem, T. Logico-combinatorial investigations in the satisfiability or provability of mathematical propositions: A simplified proof of a theorem by l. löwenheim and generalizations of the theorem. *From Frege to Gödel. A Source Book in Mathematical Logic*, 1931:252–263, 1879.
- Son, K., Kim, D., Kang, W. J., Hostallero, D. E., and Yi, Y. QTRAN: Learning to factorize with transformation for cooperative multi-agent reinforcement learning. In Chaudhuri, K. and Salakhutdinov, R. (eds.), *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pp. 5887–5896. PMLR, 09–15 Jun 2019. URL <https://proceedings.mlr.press/v97/son19a.html>.
- Sunehag, P., Lever, G., Gruslys, A., Czarnecki, W. M., Zambaldi, V., Jaderberg, M., Lanctot, M., Sonnerat, N., Leibo, J. Z., Tuyls, K., and Graepel, T. Value-decomposition networks for cooperative multi-agent learning, 2017. URL <https://arxiv.org/abs/1706.05296>.
- Tan, M. Multi-agent reinforcement learning: Independent vs. cooperative agents. In *Proceedings of the tenth international conference on machine learning*, pp. 330–337, 1993. URL <http://dx.doi.org/10.1016/B978-1-55860-307-3.50049-6>.

- Vamplew, P., Dazeley, R., Berry, A., Issabekov, R., and Dekker, E. Empirical evaluation methods for multiobjective reinforcement learning algorithms. *Machine learning*, 84:51–80, 2011. URL <https://dl.acm.org/doi/10.1007/s10994-010-5232-5>.
- Wang, Y., Bozkurt, A. K., and Pajic, M. Reinforcement learning with temporal logic constraints for partially-observable markov decision processes, 2021. URL <https://arxiv.org/abs/2104.01612>.
- Winter, S. and Zimmermann, M. Tracy, traces, and transducers: computable counterexamples and explanations for hyperltl model-checking, 2025. URL <https://doi.org/10.1007/s00236-025-00499-7>.
- Xu, Z. and Topcu, U. Transfer of temporal logic formulas in reinforcement learning. In *IJCAI: proceedings of the conference*, volume 28, pp. 4010. NIH Public Access, 2019. URL <http://dx.doi.org/10.24963/ijcai.2019/557>.
- Yuan, L. Z., Hasanbeig, M., Abate, A., and Kroening, D. Modular deep reinforcement learning with temporal logic specifications, 2019. URL <https://arxiv.org/abs/1909.11591>.

A. Proofs

A.1. Proof of Theorem 5.1

Proof. Let $\mathcal{M}$ be a POMDP and let $\mathbf{Skolem}(\varphi)$ denote the Skolemized form of the HyperLTL formula φ , written as

$$\mathbf{Skolem}(\varphi) = \exists \mathbf{f}_i(\tau_{i_1}, \dots, \tau_{i_{|\mathbb{Q}^\forall|}}) \forall \tau_j. \mathbf{Skolem}(\psi),$$

for each $i \in \mathbb{Q}^\exists$ and $j \in \mathbb{Q}^\forall$. Assume that the tuple of policies $\langle \pi_i^* \rangle_{i \in \mathbb{Q}^\exists} \sqcup \langle \pi_j^* \rangle_{j \in \mathbb{Q}^\forall}$ optimizes the robustness-based objective induced by $\mathbf{Skolem}(\varphi)$.

Since Φ is path-consistent, for any history h generated by a latent path ζ , $\hat{\zeta} \triangleq \arg \max_{\zeta \in \mathcal{Z}^*} \Phi(h, \zeta)$ coincides with the actual path ζ that generated h . Consequently, for any $k \geq 0$, $\text{Tr}(\hat{\zeta}_{[0:k]}) = \text{Tr}(\zeta_{[0:k]})$.

Fix an arbitrary $k \geq 0$. The zipped path induces a trace assignment for the variables of φ as follows.

- For each universally quantified trace variable τ_j , where $j \in \mathbb{Q}^\forall$,

$$\tau_j \mapsto \text{Tr}((\arg \max_{\zeta \in \mathcal{Z}^*} \Phi(h_j, \zeta))_{[0:k]}).$$

- For each existentially quantified trace variable τ_i , where $i \in \mathbb{Q}^\exists$,

$$\tau_i \mapsto \mathbf{f}_i \left(\text{Tr}((\arg \max_{\zeta \in \mathcal{Z}^*} \Phi(h_{i_1}, \zeta))_{[0:k]}), \dots, \text{Tr}((\arg \max_{\zeta \in \mathcal{Z}^*} \Phi(h_{i_{|\mathbb{Q}^\forall|}}, \zeta))_{[0:k]}) \right).$$

By construction of Skolemization, each Skolem function $\mathbf{f}_i$ depends only on trace variables corresponding to universally quantified indices preceding i . Moreover, if $i_\ell = j$ for some ℓ , then the corresponding trace prefixes coincide. Since the given policy tuple optimizes $\mathbf{Skolem}(\varphi)$, it maximizes the probability that the robustness value

$$\rho(\text{zip}(\cdot), \mathbf{Skolem}(\psi))$$

converges to $\rho_{\max}$. By the quantitative semantics of LTL, convergence of robustness to $\rho_{\max}$ is equivalent to satisfaction of the inner LTL formula $\mathbf{Skolem}(\psi)$. Therefore, the induced trace assignment maximizes the probability of satisfying $\mathbf{Skolem}(\varphi)$.

Lifting to the original HyperLTL formula. Since Skolemization preserves satisfiability of HyperLTL formulas, and existentially quantified traces are instantiated as Skolem witnesses depending only on preceding universal traces, maximizing the robustness-based objective induced by $\mathbf{Skolem}(\varphi)$ yields a tuple of policies that maximizes the probability of satisfying the original HyperLTL formula φ in $\mathcal{M}$. Hence, the tuple of policies $\langle \pi_i^* \rangle_{i \in \mathbb{Q}^\exists} \sqcup \langle \pi_j^* \rangle_{j \in \mathbb{Q}^\forall}$ optimizes the probability of satisfying φ in $\mathcal{M}$. $\square$

A.2. Proof of Theorem 5.3

Proof. Given a POMDP $\mathcal{M}$ and a HyperLTL formula φ , let $\mathbf{Skolem}(\varphi)$ be the Skolemized form of φ , and let the robustness values $\rho(\text{zip}(\cdot), \mathbf{Skolem}(\psi))$ be used as the reward signal by the CTDE algorithm. The CTDE algorithm optimizes a joint action-value function $\mathbf{Q}(\mathbf{h}, \mathbf{a})$ that evaluates the expected discounted return under the robustness-based reward signal. Hence, an optimal joint policy π^* computed as: $\mathbf{Q}^{\pi^*}(\mathbf{h}, \mathbf{a}) = \max_{\pi} \mathbf{Q}^{\pi}(\mathbf{h}, \mathbf{a})$,

We assume that $\mathbf{Q}$ satisfies the IGM property. Then there exist individual action-value functions $\langle Q_i \rangle_{i \in \{1, \dots, n\}}$ such that for every joint history $\mathbf{h} = \langle h_\ell \rangle_{\ell \in \{1, \dots, n\}}$,

$$\arg \max_a \mathbf{Q}(\mathbf{h}, \mathbf{a}) = \left\langle \arg \max_{a_\ell} Q_\ell(h_\ell, a_\ell) \right\rangle_{\ell \in \mathbb{Q}^\exists \cup \mathbb{Q}^\forall}.$$

Hence we get optimal decentralized policies from:

$$Q_\ell^{\pi^*}(h_\ell, a_\ell) = \max_{\pi} Q_\ell^{\pi}(h_\ell, a_\ell), \quad \text{for each } \ell \in \mathbb{Q}^\exists \cup \mathbb{Q}^\forall.$$

By the IGM equality above, the induced joint greedy action $\langle \pi_\ell^*(h_\ell) \rangle_\ell$ attains a joint maximizer of $\mathbf{Q}(\mathbf{h}, \cdot)$ for given $\mathbf{h}$; therefore the induced joint policy is optimal w.r.t. $\mathbf{Q}$. It is important to note that we the input of the policies with the existential quantifier gets the history associated with

By construction, the CTDE algorithm uses the robustness values $\rho(\text{zip}(\cdot), \text{Skolem}(\psi))$ as the reward signal. Hence, the learned joint action–value function $\mathbf{Q}$ optimizes the same objective as (3), i.e., it maximizes (over tuples of policies) the probability that the robustness-value of the zipped traces w.r.t. $\text{Skolem}(\psi)$ converges to $\rho_{\max}$. Therefore, any policy tuple induced by the decentralized policies $\langle \pi_i^* \rangle_{i \in \mathbb{Q}^\exists}$ and $\langle \pi_j^* \rangle_{j \in \mathbb{Q}^\forall}$ is an optimizer of (3), and thus optimizes the satisfaction probability of $\text{Skolem}(\varphi)$. Note that our theoretical guarantees assume that, during decentralized execution, $f_{\text{enc-dec}}$ accurately reconstructs the relevant universally quantified trace prefixes required by the Skolem witnesses. $\square$

B. Continuation of Problem Statement Running Example

Consider the POMDP in Fig. 2. We use the following HyperLTL formula:

$$\varphi_{\text{exp2}} \triangleq \forall \tau_1 \forall \tau_2. \text{Dist}(\text{pos}_{\tau_1}, \text{pos}_{\tau_2}) < 3 \ \mathcal{U} \ (\text{pos}_{\tau_1} = i)$$

FF, corresponding to τ_1 , samples histories using policy π_1 , $\mathcal{H}_{\tau_1} = \{h_1^1, h_1^2\}$:

$$\begin{aligned} h_1^1 &: \langle a \rangle \xrightarrow{\text{R}} \langle b \rangle \xrightarrow{\text{R}} \langle c, \textcolor{red}{c} \rangle \xrightarrow{\text{U}} \langle f, \textcolor{blue}{f}, \textcolor{red}{f} \rangle \xrightarrow{\text{U}} \langle i, \textcolor{red}{i} \rangle \\ h_1^2 &: \langle a \rangle \xrightarrow{\text{R}} \langle b \rangle \xrightarrow{\text{R}} \langle e \rangle \xrightarrow{\text{U}} \langle h \rangle \xrightarrow{\text{R}} \langle i, \textcolor{red}{i} \rangle \end{aligned}$$

Med, corresponding to τ_2 , samples histories using policy π_2 , $\mathcal{H}_{\tau_2} = \{h_2^1, h_2^2\}$:

$$\begin{aligned} h_2^1 &: \langle a \rangle \xrightarrow{\text{U}} \langle d \rangle \xrightarrow{\text{U}} \langle g, \textcolor{blue}{g} \rangle \xrightarrow{\text{D}} \langle d \rangle \xrightarrow{\text{R}} \langle d \rangle \\ h_2^2 &: \langle a \rangle \xrightarrow{\text{U}} \langle b \rangle \xrightarrow{\text{L}} \langle e \rangle \xrightarrow{\text{R}} \langle f, \textcolor{blue}{f}, \textcolor{red}{f} \rangle \xrightarrow{\text{U}} \langle i, \textcolor{red}{i} \rangle \end{aligned}$$

Assume that Φ is path-consistent. Then we can construct the paths associated with $\mathcal{H}_{\tau_1}$ and $\mathcal{H}_{\tau_2}$ as $\mathcal{Z}_{\tau_1} = \{\arg \max_{\zeta \in \mathcal{Z}^*} \Phi(h_1^1, \zeta), \arg \max_{\zeta \in \mathcal{Z}^*} \Phi(h_1^2, \zeta)\}$ and $\mathcal{Z}_{\tau_2} = \{\arg \max_{\zeta \in \mathcal{Z}^*} \Phi(h_2^1, \zeta), \arg \max_{\zeta \in \mathcal{Z}^*} \Phi(h_2^2, \zeta)\}$, respectively. We now compute the probability of satisfying φ_{exp2} using $\mathcal{Z}_{\tau_1}$ and $\mathcal{Z}_{\tau_2}$; the evaluation ranges over all pairwise combinations of paths in $\mathcal{Z}_{\tau_1} \times \mathcal{Z}_{\tau_2}$:

$$\begin{aligned} \text{Traces}(\langle \{\arg \max_{\zeta \in \mathcal{Z}^*} \Phi(h_1^1, \zeta)\}, \{\arg \max_{\zeta \in \mathcal{Z}^*} \Phi(h_2^1, \zeta)\} \rangle) &\not\models \varphi_{\text{exp2}} \quad \text{Traces}(\langle \{\arg \max_{\zeta \in \mathcal{Z}^*} \Phi(h_1^1, \zeta)\}, \{\arg \max_{\zeta \in \mathcal{Z}^*} \Phi(h_2^2, \zeta)\} \rangle) \models \varphi_{\text{exp2}} \\ \text{Traces}(\langle \{\arg \max_{\zeta \in \mathcal{Z}^*} \Phi(h_1^2, \zeta)\}, \{\arg \max_{\zeta \in \mathcal{Z}^*} \Phi(h_2^1, \zeta)\} \rangle) &\models \varphi_{\text{exp2}} \quad \text{Traces}(\langle \{\arg \max_{\zeta \in \mathcal{Z}^*} \Phi(h_1^2, \zeta)\}, \{\arg \max_{\zeta \in \mathcal{Z}^*} \Phi(h_2^2, \zeta)\} \rangle) \models \varphi_{\text{exp2}} \end{aligned}$$

Out of the four possible pairs in $\mathcal{Z}_{\tau_1} \times \mathcal{Z}_{\tau_2}$, only three satisfy φ_{exp2} , yielding a satisfaction probability of 0.75.

C. More Technical Details for Sec. 5

C.1. Details on Sec. 5.1

Let $\varphi = \mathbb{Q}_1 \tau_1 \dots \mathbb{Q}_n \tau_n$. $\psi(\tau_1, \dots, \tau_n)$ be a HyperLTL formula. The Skolemized formula has the form:

$$\text{Skolem}(\varphi) = \underbrace{\exists \mathbf{f}_i(\tau_{i_1}, \dots, \tau_{i_{|\mathbb{Q}_i^\forall|}})}_{\text{for each } i \in \mathbb{Q}^\exists} \cdot \underbrace{\forall \tau_j}_{\text{for each } j \in \mathbb{Q}^\forall} \cdot \text{Skolem}(\psi)$$

Using the Transformation , we reformulate the problem in Fig. 3. We begin by defining the *image* of a Skolem function $\mathbf{f}_i$ as:

$$\text{Img}(\mathbf{f}_i) \triangleq \{\mathbf{f}_i(t_{i_1}, \dots, t_{i_{|\mathbb{Q}_i^\forall|}}) \mid t_{i_j} \in \text{Traces}(\bigcup_{h \in (\mathcal{H}_{i_j} \sim \mathcal{D}_{\pi_{i_j}})} \{\arg \max_{\zeta \in \mathcal{Z}^*} \Phi(h, \zeta)\}), j \in \mathbb{Q}_i^\forall\}$$

That is, $\text{Img}(\mathbf{f}_i)$ is the set of all traces obtained by mapping the preceding universally quantified traces τ_{i_j} , where each t_{i_j} ranges over the trace set derived from the corresponding history set $\mathcal{H}_{i_j}$. Now, let us use $\bowtie$ to indicate that the collection of

trace sets is ordered with respect to their path indices. That is, given two tuples of sets of traces $\mathcal{T}_1$ and $\mathcal{T}_2$, applying $\bowtie$ yields an ordered tuple $\mathcal{T}_1 \bowtie \mathcal{T}_2 \triangleq \langle \text{Traces}(\mathcal{Z}_{\tau_x}) \rangle_{x \in \{1, \dots, n\}}$, where each $\text{Traces}(\mathcal{Z}_{\tau_x})$ belongs to either $\mathcal{T}_1$ or $\mathcal{T}_2$. Returning to the problem in Fig. 3, given a POMDP $\mathcal{M}$ and a HyperLTL formula φ , our goal is to compute: a tuple of Skolem witnesses $\langle \mathbf{f}_i \rangle_{i \in \mathbb{Q}^\exists}$, and a tuple of optimal policies $\langle \pi_j^* \rangle_{j \in \mathbb{Q}^\forall}$, such that:

$$\langle \pi_j^* \rangle_j \in \left[\arg \max_{\langle \pi_j \rangle} \mathbb{P} \left[\langle \text{Img}(\mathbf{f}_i) \rangle \bowtie \langle \text{Traces}(\bigcup_{h \in (\mathcal{H}_{i_j} \sim \mathcal{D}_{\pi_{i_j}})} \{\arg \max_{\zeta \in \mathcal{Z}^*} \Phi(h, \zeta)\}) \rangle \models \mathbf{Skolem}(\varphi) \right] \right]_{i \in \mathbb{Q}^\exists, j \in \mathbb{Q}^\forall}$$

The meaning of above formula is that the policy tuple $\langle \pi_j^* \rangle$ maximizes the probability that the ordered collection consisting of (1) the generated traces of the universal quantifiers $\langle \text{Traces}(\mathcal{Z}_{\tau_j}) \rangle_{j \in \mathbb{Q}^\forall}$ and (2) the Skolem witness for all existential quantifiers $\langle \text{Img}(\mathbf{f}_i) \rangle_{i \in \mathbb{Q}^\exists}$ jointly satisfies $\mathbf{Skolem}(\varphi)$. In the reformulated problem, policies are synthesized solely for universally quantified traces, whereas Skolem functions are learned for existentially quantified traces to serve as witnesses of optimality.

C.2. Details on Sec. 5.2

We reformulate the HyperLTL satisfaction problem as an optimization problem under its quantitative semantics, where the Skolemized formula $\mathbf{Skolem}(\varphi)$ is evaluated on tuples of histories $\langle h_1, \dots, h_n \rangle$ sampled from the POMDP $\mathcal{M}$.

Robustness for a Single Trace. Let $\mathbb{R}$ denote the set of real numbers and let Ψ denote the set of all LTL formulas. Let $f : 2^{\text{AP}} \rightarrow \mathbb{R}$ be a valuation function that assigns real values to sets of atomic propositions. Let $\mathbb{R}$ be the set of real numbers, Ψ the set of all LTL formulas, and $f : 2^{\text{AP}} \rightarrow \mathbb{R}$ be a *valuation function* assigning real values to sets of atomic propositions. For a state $s \in S$ of $\mathcal{M}$, quantitative semantics are defined by predicates of the form $f(L(s)) < c$, where c is a user-specified threshold (see Fig. 9). Different values of c can be assigned to different segments of the inner LTL formula to emphasize the relative priority of those segments. The robustness function $\rho : \text{Traces}(\mathcal{Z}^*) \times \Psi \rightarrow \mathbb{R}$ assigns a real-valued score to a finite trace and an LTL formula, bounded by $\rho_{\max}$ and $\rho_{\min}$. For a given trace, higher ρ values correspond to greater robustness in satisfying ψ , while lower values indicate weaker satisfaction or possible violation. Formally, given an LTL formula ψ and a POMDP $\mathcal{M}$, we formulate the problem of synthesizing a policy π^* as: Formally, given an LTL formula ψ and a POMDP $\mathcal{M}$, we formulate the policy synthesis problem as:

$$\pi^* \in \arg \max_{\pi} \mathbb{P}_{h \sim \mathcal{D}_{\pi}} \left[\rho(\text{Tr}((\arg \max_{\zeta \in \mathcal{Z}^*} \Phi(h, \zeta))_{[0:k]}), \psi) \xrightarrow{*} \rho_{\max} \right],$$

where $\xrightarrow{*}$ means convergence. I.e., π^* maximizes the probability that paths induced by histories under π satisfy ψ .

Robustness for a Tuple of Traces. Next, to compute robustness over multiple (universally quantified) traces, we define a zip operator as follows: that performs pointwise aggregation over a tuple of traces. Given a tuple of finite traces $\langle t_\ell \rangle_{\ell \in \{1, \dots, n\}}$, where each $t_\ell = t_\ell(0)t_\ell(1) \dots t_\ell(m_\ell)$, we define their *zipped trace* as:

$$\text{zip}(\langle t_\ell \rangle_{\ell \in \{1, \dots, n\}}) \triangleq \langle t_\ell(0) \rangle_{\ell \in \{1, \dots, n\}} \dots \langle t_\ell(k) \rangle_{\ell \in \{1, \dots, n\}}$$

Given an LTL formula ψ , a tuple of paths $\langle \zeta_1, \zeta_2, \dots, \zeta_n \rangle$ is more likely to satisfy ψ if the robustness value of $\text{zip}(\langle \text{Tr}(\zeta_1[0:k_1]), \text{Tr}(\zeta_2[0:k_2]), \dots, \text{Tr}(\zeta_n[0:k_n]) \rangle)$ converges to $\rho_{\max}$ with respect to ψ for some $k_1, \dots, k_n$, where $0 \leq k_\ell \leq |\zeta_\ell|$ for each $1 \leq \ell \leq n$.

Thus, the optimization problem of computing a tuple of policies $\langle \pi_1^*, \pi_2^*, \dots, \pi_n^* \rangle$ that maximizes robustness can be stated as:

$$\langle \pi_\ell^* \rangle_\ell \in \left[\arg \max_{\langle \pi_\ell \rangle} \mathbb{P} \left[\rho(\text{zip}(\text{Tr}((\arg \max_{\zeta \in \mathcal{Z}^*} \Phi(h_\ell \sim \mathcal{D}_{\pi_\ell}, \zeta))_{[0:k_\ell]}))), \psi) \xrightarrow{*} \rho_{\max} \right] \right]_{\ell \in \{1, \dots, n\}}$$

This transformation highlights that LTL specifications are implicitly universally quantified in the single-trace setting.

Robustness for Skolemized HyperLTL. We now address the optimization problem induced by an alternating HyperLTL formula. Let $\langle t_i \rangle_{i \in I}$ and $\langle t_j \rangle_{j \in J}$ be two trace tuples, where $I \cup J = \{1, \dots, n\}$ and $I \cap J = \emptyset$. To preserve index order when combining them, we use $\sqcup$, which produces a single tuple ordered by path indices. In particular, applying $\sqcup$ produces

$$\begin{aligned}
 \rho(\text{Tr}(\zeta_{[\ell:k]}), \psi) &= \rho_{\min} \text{ if } \text{Tr}(\zeta_{[\ell:]}) = \epsilon \text{ and } \rho(\text{Tr}(\zeta_{[\ell:k]}), \psi) \text{ otherwise.} \\
 \rho(\text{Tr}(\zeta_{[\ell:k]}), \text{true}) &= \rho_{\max} \\
 \rho(\text{Tr}(\zeta_{[\ell:k]}), f(L(s_\ell) < c)) &= c - f(L(s_\ell)) \\
 \rho(\text{Tr}(\zeta_{[\ell:k]}), \neg\psi) &= -\rho(\text{Tr}(\zeta_{[\ell:k]}), \psi) \\
 \rho(\text{Tr}(\zeta_{[\ell:k]}), \bigcirc\psi) &= \rho(\text{Tr}(\zeta_{[\ell+1:k]}), \psi) \text{ if } (k > \ell). \\
 \rho(\text{Tr}(\zeta_{[\ell:k]}), \Box\psi) &= \min_{i \in [\ell, k]} \rho(\text{Tr}(\zeta_{[i:k]}), \psi) \\
 \rho(\text{Tr}(\zeta_{[\ell:k]}), \Diamond\psi) &= \max_{i \in [\ell, k]} \rho(\text{Tr}(\zeta_{[i:k]}), \psi) \\
 \rho(\text{Tr}(\zeta_{[\ell:k]}), \psi_1 \wedge \psi_2) &= \min(\rho(\text{Tr}(\zeta_{[\ell:k]}), \psi_1), \rho(\text{Tr}(\zeta_{[\ell:k]}), \psi_2)) \\
 \rho(\text{Tr}(\zeta_{[\ell:k]}), \psi_1 \vee \psi_2) &= \max(\rho(\text{Tr}(\zeta_{[\ell:k]}), \psi_1), \rho(\text{Tr}(\zeta_{[\ell:k]}), \psi_2)) \\
 \rho(\text{Tr}(\zeta_{[\ell:k]}), \psi_1 \mathcal{U} \psi_2) &= \max_{i \in [\ell, k]} \left(\min \left(\rho(\text{Tr}(\zeta_{[i:k]}), \psi_2), \min_{j \in [\ell, i]} \rho(\text{Tr}(\zeta_{[j:i]}), \psi_1) \right) \right)
 \end{aligned}$$

Figure 9. Quantitative semantics for LTL, adapted from (Hsu et al., 2025; Li et al., 2017).

a tuple $\text{Tr}(\zeta_1) < \text{Tr}(\zeta_2) < \dots < \text{Tr}(\zeta_n)$, where $\text{Tr}(\zeta_x) < \text{Tr}(\zeta_y)$ denotes $x < y$. Based on this construction, satisfaction of the inner LTL body ψ is defined over tuples of histories. For each $i \in \mathbb{Q}^\exists$ and $j \in \mathbb{Q}^\forall$, a tuple $\langle h_1, \dots, h_n \rangle$ satisfies ψ if and only if:

$$\rho\left(\text{zip}\left(\langle \text{Tr}(\arg \max_{\zeta \in \mathcal{Z}^*} \Phi(h_i, \zeta))_{[0:k_i]} \rangle \sqcup \langle \text{Tr}(\arg \max_{\zeta \in \mathcal{Z}^*} \Phi(h_j, \zeta))_{[0:k_j]} \rangle\right), \text{Skolem}(\psi)\right) \xrightarrow{*} \rho_{\max}$$

Therefore, we formulate the optimization problem for the Skolemized HyperLTL formula as follows:

$$\langle \pi_i^* \rangle \sqcup \langle \pi_j^* \rangle \in \left[\arg \max_{\langle \pi_i \rangle \sqcup \langle \pi_j \rangle} \mathbb{P} \left[\rho\left(\text{zip}\left(\langle \text{Tr}((\arg \max_{\zeta \in \mathcal{Z}^*} \Phi(h_i \sim \mathcal{D}_{\pi_i}, \zeta))_{[0:k_i]} \rangle \sqcup \langle \text{Tr}((\arg \max_{\zeta \in \mathcal{Z}^*} \Phi(h_j \sim \mathcal{D}_{\pi_j}, \zeta))_{[0:k_j]} \rangle\right), \text{Skolem}(\psi)\right) \xrightarrow{*} \rho_{\max} \right] \right]_{i \in \mathbb{Q}^\exists, j \in \mathbb{Q}^\forall}$$

D. Experimental Setup and Implementation Details

In this section, we detail the implementation, architecture, and hyperparameters used in our learning process.

D.1. Details on MARL used in HYPOLE

In our experiments, we use the implementations of QMIX, VDN, QTRAN, and IQL provided by <https://github.com/oxwhirl/pymarl>. For QMIX, we set `epsilon_start=1.0`, `epsilon_finish=0.05`, `epsilon_anneal_time=50000`, `buffer_size=5000`, `target_update_interval=200`, `mixing_embed_dim=64`, `opt_loss=1`, and `nopt_min_loss=0.1`. For VDN, we set `epsilon_start=1.0`, `epsilon_finish=0.05`, `epsilon_anneal_time=50000`, `buffer_size=5000`, and `target_update_interval=200`. For QTRAN, we set `epsilon_start=1.0`, `epsilon_finish=0.05`, `epsilon_anneal_time=50000`, `buffer_size=5000`, `target_update_interval=200`, `mixing_embed_dim=64`, `opt_loss=1`, and `nopt_min_loss=0.1`. Finally, for IQL, we set `epsilon_start=1.0`, `epsilon_finish=0.05`, `epsilon_anneal_time=50000`, `buffer_size=5000`, and `target_update_interval=200`.

Table 1. SMAC maps used in evaluation, including unit configurations and HyperLTL formulas used in our experiments.

Map Name	Ally Units	Enemy Units	Formula	Quantifiers
3m	3 Marines	3 Marines	$\varphi_{\text{focus-fire}}, \varphi_{\text{defense}}, \varphi_{\text{bad}}$	$\forall^*$
8m	3 Marines	3 Marines	$\varphi_{\text{focus-fire}}$	$\forall^*$
2s3z	2 Stalkers & 3 Zealots	2 Stalkers & 3 Zealots	$\varphi_{\text{focus-fire}}$	$\forall^*$
3s5z	3 Stalkers & 5 Zealots	3 Stalkers & 5 Zealots	$\varphi_{\text{focus-fire}}$	$\forall^*$
MMM	1 Med & 2 Marauders & 7 Marines	1 Med & 2 Marauders & 7 Marines	φ_{medivac}	$\forall^*, \forall^* \exists$
5m_vs_6m	5 Marines	6 Marines	$\varphi_{\text{focus-fire}}, \varphi_{\text{kite}}$	$\forall^*$
MMM2	1 Med, 2 Marauders & 7 Marines	1 Med, 2 Marauders & 8 Marines	φ_{medivac}	$\forall^*, \forall^* \exists$
bane_vs_bane	20 Zerglings & 4 Banelings	20 Zerglings & 4 Banelings	$\varphi_{\text{focus-fire}}$	$\forall^*$
corridor	6 Zealots	24 Zerglings	$\varphi_{\text{corridor}}$	$\forall^*$

D.2. Details on $f_{\text{enc-dec}}, \mathbf{f}, \Omega$ used in HYPOLE

$f_{\text{enc-dec}}$ is a feedforward encoder-decoder (autoencoder) model implemented in PyTorch. Both the encoder and decoder are three-layer MLPs composed of Linear layers. We use ReLU activations after each hidden layer, and apply a final Tanh activation at the decoder output to bound predictions to $[-1, 1]$, matching the normalized target range. The first hidden layer has width equal to the observation dimension, and the second hidden layer has 256 units; the remaining layer sizes follow this encoder-decoder symmetry. We optimize the model using Adam with learning rate 10^{-3} and train it by minimizing mean squared error (MSE).

To train $f_{\text{enc-dec}}$ model, we maintain a buffer $\mathcal{D}$ containing the 10,000 most recent agent with existential quantifier observations and their corresponding global states at the same timestep. After each episode, we uniformly sample a mini-batch of size 32 from the pair buffer and perform one gradient update step. To reduce unnecessary computation, we apply a decaying update schedule based β on the current training loss. When the loss exceeds 9×10^{-5} , we update $f_{\text{enc-dec}}$ every 50 environment steps. When the loss is between 9×10^{-5} and 9×10^{-6} , we update every 100 steps. Once the loss drops below 9×10^{-6} , we update every 1000 steps.

Next, for each agent with an existential quantifier $i \in \mathbb{Q}^{\exists}$, we use an RNN to approximate $\Omega(\mathbf{f}_i(\cdot))$ and obtain a history representation $\hat{h}_i$. Basically, for agents associated with existential quantifiers, instead of feeding only their local observations o_i into the RNN module, we first predict the global state sequences of the preceding universally quantified agents, denoted by $(\hat{s}_{i_1[0:k]}, \dots, \hat{s}_{i_{|\mathbb{Q}^{\forall}|}[0:k]})$. This predicted information is then incorporated into the input of the RNN, producing a history representation $\hat{h}_i$. The resulting representation contains information about both agent i and the preceding universally quantified agents. We then use $\hat{h}_i$ to construct the agent’s utility function and decentralized policy π_i . For the remaining components of CTDE, we follow the standard PyMARL implementation. It is important to note that the RNN approximating $\Omega(\mathbf{f}_i(\cdot))$ is trained jointly with CTDE.

D.3. Details on Experimental Setup and Training

We ran training on three machines. The first machine has an AMD EPYC 7742 64-core CPU and three NVIDIA RTX A6000 GPUs. The second is a shared server with an Intel Xeon Platinum 8268 CPU and four NVIDIA Quadro RTX 8000 GPUs; on this machine, CPU resources are limited due to sharing (GPU resources are not). The third machine has an AMD EPYC 7742 64-core CPU and three NVIDIA RTX PRO 6000 Blackwell Max-Q Workstation Edition GPUs. For SMAC and MessySMAC, we train each agent for 10k environment steps and evaluate every checkpoint over 32 test episodes. All reported SMAC and MessySMAC results are obtained in `test_mode`. For WildFire, we train each agent for 5k environment steps and evaluate every checkpoint over 30 test episodes. All SMAC and Wildfire experiments were conducted over ten independent runs with different random seeds, while MessySMAC experiments were conducted over five independent runs.

E. Details on SMAC and MessySMAC Benchmark

We evaluate on the StarCraft II Multi-Agent Challenge (SMAC) benchmark <https://github.com/oxwhirl/smac>. Table 1 lists the scenarios used in our experiments, together with the HyperLTL formulas we evaluate and their corresponding quantifier structures. For baseline comparisons, we use the shaped reward provided by SMAC, which combines hit-point

Table 2. WildFire maps used in evaluation, including agents configurations and HyperLTL formulas used in our experiments.

Map Name	Grid Size	Agents	Objects	Formula
5P_1F1V	5×5	1 FF, 1 Med	1 Victim 1 Fire Zone	$\varphi_{\text{wildfire}}$
5P_3F2V	5×5	1 FF, 1 Med	2 Victim 3 Fire Zone	$\varphi_{\text{wildfire}}$
8P_3F2V	8×8	1 FF, 1 Med	2 Victim 3 Fire Zone	$\varphi_{\text{wildfire}}$
10P_3F2V	10×10	1 FF, 1 Med	2 Victim 3 Fire Zone	$\varphi_{\text{wildfire}}$

damage, enemy kills, and an additional bonus for winning the battle. SMAC also includes a sparse reward (+1 for winning and -1 for losing an episode); however, in our preliminary experiments the sparse reward performed substantially worse than the shaped reward. Therefore, we use the shaped reward for all baseline results.

We mostly use the same configuration for MessySMAC. However, this benchmark adds observation stochasticity ϕ and randomized initialization with K random steps before each episode starts. For the 3m map, we use ($K = 10, \phi = 0.15$). For the 8m, 2s3z, 3s5z, and MMM maps, we use ($K = 12, \phi = 0.15$).

F. Details on Wildfire Benchmark

We extend the Wildfire scenario from Hsu et al. (2025) to a partially observable setting. The environment contains two agent types: a firefighter drone FF  and a medical drone Med  (see Fig. 2). The agents must extinguish fires and rescue victims within a fixed time while maintaining a safe separation distance. In this setting, each agent observes only entities within a fixed sight range. We use the maps listed in Table 2. The global state includes all agent features and object (fire/victim) features, the last action, and the time step. Agent and object features are represented by centered coordinates: $(x - \text{center}_x, y - \text{center}_y)$, normalized by the maximum horizontal and vertical distances, respectively. The last-action vector encodes each agent’s previous action. Each agent’s observation consists of its move features, own features, the time step, and the features of agents and objects within its sight range. Move features specify the set of available actions. The action space is {Up, Down, Left, Right}. For each observed entity, we include relative position (normalized by sight range), distance to the entity (normalized by sight range), the entity type ID, and their last action.

We introduce three reasonable baseline reward functions R_{Wild_1} , R_{Wild_2} , R_{Wild_3} . The functions are as follows:

$$R_{\text{Wild}_1} = \begin{cases} 20 & \text{Victim Saved} \\ 20 & \text{Fire Ext} \\ -10 & \text{Out of Range} \\ -10 & \text{Med in Fire} \end{cases} \quad R_{\text{Wild}_2} = \begin{cases} 20 & \text{Victim Saved} \\ 20 & \text{Fire Ext} \\ -10 & \text{Out of Range} \\ -10 & \text{Med in Fire} \\ +100 & \text{Mission Accomplish} \end{cases} \quad R_{\text{Wild}_3} = \begin{cases} 20 & \text{Victim Saved} \\ 20 & \text{Fire Ext} \\ -10 & \text{Out of Range} \\ -10 & \text{Med in Fire} \\ +100 & \text{Mission Accomplish} \\ -50 & \text{Mission failed} \end{cases}$$

G. HyperLTL Formulas

We provide all HyperLTL formulas used in the SMAC and MessySMAC experiments, namely $\varphi_{\text{focus-fire}}$, φ_{medivac} , φ_{kite} , $\varphi_{\text{corridor}}$, φ_{defense} , and φ_{bad} , as well as $\varphi_{\text{wildfire}}$ for the WildFire benchmark.

$$\begin{aligned}
 \varphi_{\text{focus-fire}} \triangleq & \overbrace{\forall \tau_1, \dots, \tau_n}^{\text{Ally Agents}} \cdot \left[\overbrace{\left(\left(\text{dist}(\text{pos}_{\tau_1}, \text{pos_enm_1}) < \text{s_range}_{\tau_1} \wedge \dots \wedge \text{dist}(\text{pos}_{\tau_n}, \text{pos_enm_1}) < \text{s_range}_{\tau_n} \right) \vee \right.}^{\text{Focused Shooting}} \\
 & \dots \vee \left(\text{dist}(\text{pos}_{\tau_1}, \text{pos_enm_m}) < \text{s_range}_{\tau_1} \wedge \dots \wedge \text{dist}(\text{pos}_{\tau_n}, \text{pos_enm_m}) < \text{s_range}_{\tau_n} \right) \wedge \\
 & \left. \underbrace{\left((\text{health_enm_1} < \text{health_enm_prev_1}) \vee \dots \vee (\text{health_enm_m} < \text{health_enm_prev_m}) \right)}_{\text{Damaging Enemies Health/Shield Bar}} \right) \\
 & \left. \underbrace{\mathcal{U} \# \text{enm_dead} > \# \text{enm_dead_prev}}_{\text{Eliminating Enemies}} \right] \mathcal{U} \text{ Win}
 \end{aligned}$$

$$\begin{aligned}
 \varphi_{\text{medivac}} \triangleq & \overbrace{\forall \tau_1, \dots, \tau_{n-1}}^{\text{Ally Agents}} \overbrace{\forall \tau_n / \exists \tau_n}^{\text{Medivac}} \cdot \\
 & \left[\overbrace{\left(\left(\text{dist}(\text{pos}_{\tau_1}, \text{pos_enm_1}) < \text{s_range}_{\tau_1} \wedge \dots \wedge \text{dist}(\text{pos}_{\tau_{n-1}}, \text{pos_enm_1}) < \text{s_range}_{\tau_{n-1}} \right) \vee \right.}^{\text{Focused Shooting}} \\
 & \dots \vee \left(\text{dist}(\text{pos}_{\tau_1}, \text{pos_enm_m}) < \text{s_range}_{\tau_1} \wedge \dots \wedge \text{dist}(\text{pos}_{\tau_{n-1}}, \text{pos_enm_m}) < \text{s_range}_{\tau_{n-1}} \right) \wedge \\
 & \underbrace{\left(\text{dist}(\text{pos}_{\tau_n}, \text{Min_Health}(\tau_1, \dots, \tau_{n-1})) < \text{h_range}_{\tau_n} \right)}_{\text{Medivac Close to Agent With Min Health}} \wedge \\
 & \underbrace{\left(\left(\sum_{i=1}^{n-1} \text{health_prev}_{\tau_i} - \sum_{i=1}^{n-1} \text{health}_{\tau_i} \right) < \left(\sum_{i=1}^m \text{health_enm_prev_i} - \sum_{i=1}^m \text{health_enm_i} \right) \right)}_{\text{Damaging Enemies Health/Shield Bar While Preserving Allies Health/Shield Bar}} \\
 & \left. \underbrace{\mathcal{U} \# \text{enm_dead} > \# \text{enm_dead_prev}}_{\text{Eliminating Enemies}} \right] \mathcal{U} \text{ Win}
 \end{aligned}$$

$$\begin{aligned}
 \varphi_{\text{kite}} \triangleq & \overbrace{\forall \tau_1, \dots, \tau_n}^{\text{Ally Agents}} \cdot \left[\overbrace{\left((\neg \text{ready}_{\tau_1} \vee \left(\bigwedge_{i=1}^m \text{dist}(\text{pos}_{\tau_1}, \text{pos_enm_i}) < \text{s_range_i} \right)) \wedge \right.}^{\text{Kiting}} \\
 & \dots \wedge (\neg \text{ready}_{\tau_n} \vee \left(\bigwedge_{i=1}^m \text{dist}(\text{pos}_{\tau_n}, \text{pos_enm_i}) < \text{s_range_i} \right)) \wedge \\
 & \left. \underbrace{\left((\text{health_enm_1} < \text{health_enm_prev_1}) \vee \dots \vee (\text{health_enm_m} < \text{health_enm_prev_m}) \right)}_{\text{Damaging Enemies Health/Shield Bar}} \right) \\
 & \left. \underbrace{\mathcal{U} \# \text{enm_dead} > \# \text{enm_dead_prev}}_{\text{Eliminating Enemies}} \right] \mathcal{U} \text{ Win}
 \end{aligned}$$

$$\begin{aligned}
 \varphi_{\text{corridor}} \triangleq & \overbrace{\forall \tau_1, \dots, \tau_n}^{\text{Ally Agents}} \cdot \left[\overbrace{\left(\left(\text{dist}(\text{pos}_{\tau_1}, \text{pos_choke_point}) < \delta \wedge \dots \wedge \text{dist}(\text{pos}_{\tau_n}, \text{pos_choke_point}) < \delta \right) \mathcal{U} \right.}^{\text{Defend in Choke Point}} \\
 & \left. \underbrace{\left((\text{health_enm_1} < \text{health_enm_prev_1}) \vee \dots \vee (\text{health_enm_m} < \text{health_enm_prev_m}) \right)}_{\text{Damaging Enemies Health/Shield Bar}} \right) \\
 & \left. \underbrace{\mathcal{U} \# \text{enm_dead} > \# \text{enm_dead_prev}}_{\text{Eliminating Enemies}} \right] \mathcal{U} \text{ Win}
 \end{aligned}$$

$$\begin{aligned}
 \varphi_{\text{defense}} &\triangleq \overbrace{\forall \tau_1, \dots, \tau_n}^{\text{Ally Agents}} \cdot \overbrace{\left(\bigwedge_{i=1}^n \bigwedge_{j=1}^m (\text{dist}(\text{pos}_{\tau_i}, \text{pos_enm_j}) > \text{o_range_enm_j}) \right)}^{\text{Running Away from Enemies}} \wedge \underbrace{\left(\bigwedge_{i=1}^n (\text{health}_{\tau_i} > \delta) \right)}_{\text{Preserve Allies Health/Shield Bar}} \\
 \varphi_{\text{bad}} &\triangleq \overbrace{\forall \tau_1, \dots, \tau_n}^{\text{Ally Agents}} \cdot \left[\overbrace{\left(((\text{dist}(\text{pos}_{\tau_1}, \text{pos_enm_1}) < \text{s_range}_{\tau_1} \wedge \dots \wedge \text{dist}(\text{pos}_{\tau_n}, \text{pos_enm_1}) < \text{s_range}_{\tau_n}) \vee \right.}^{\text{Focused Shooting}} \right. \\
 &\quad \left. \dots \vee (\text{dist}(\text{pos}_{\tau_1}, \text{pos_enm_m}) < \text{s_range}_{\tau_1} \wedge \dots \wedge \text{dist}(\text{pos}_{\tau_n}, \text{pos_enm_m}) < \text{s_range}_{\tau_n}) \right) \mathcal{U} \underbrace{\left(\# \text{enm_dead} > \# \text{enm_dead_prev} \right)}_{\text{Eliminating Enemies}} \Big] \mathcal{U} \text{Win} \\
 \varphi_{\text{wildfire}} &\triangleq \overbrace{\forall \tau_1}^{\text{Med}} \cdot \underbrace{(\forall \tau_2 / \exists \tau_2)}_{\text{FF}} \cdot \left[\left(\overbrace{\left((\text{dist}(\text{pos}_{\tau_2}, \text{pos_fire_1}) < \text{prev_dist_FF_fire_1}_{\tau_2}) \right)}^{\text{FF Get Close to Fire}} \vee \right. \right. \\
 &\quad \left. \dots \vee (\text{dist}(\text{pos}_{\tau_2}, \text{pos_fire_n}) < \text{prev_dist_fire_n}_{\tau_2}) \right) \mathcal{U} \text{Ex_fire}_{\tau_2} \Big) \wedge \\
 &\quad \left(((\text{dist}(\text{pos}_{\tau_1}, \text{pos_Vic_1}) < \text{prev_dist_Vic_1}_{\tau_1}) \vee \right. \\
 &\quad \left. \dots \vee (\text{dist}(\text{pos}_{\tau_1}, \text{pos_Vic_m}) < \text{prev_dist_Vic_m}_{\tau_1}) \right) \mathcal{U} \text{Save_Vic}_{\tau_1} \Big) \wedge \\
 &\quad \left. \text{dist}(\text{pos}_{\tau_1}, \text{pos}_{\tau_2}) < \delta \wedge \bigwedge_{i=0}^n \neg \text{pos_Fire_i}_{\tau_1} \right] \mathcal{U} \text{Mission_Accomplished}
 \end{aligned}$$

H. Additional Results

H.1. HYPOLE vs. Shaped Rewards

In addition to the maps discussed in Sec. 6, we provide results on additional SMAC scenarios. Fig. 13 summarizes performance on 3m, 2s3z, and MMM2. As shown in Fig. 13a, 3m is relatively easy and all methods achieve similar performance. In contrast, on 2s3z (Fig. 13b) we observe clear gains for HYPOLE +QTRAN (brown line) and HYPOLE +IQL (gray line) over vanilla QTRAN (purple line) and IQL (pink line). On MMM2 (Fig. 13c), where we apply φ_{medivac} within HYPOLE, HYPOLE +QMIX (orange line) clearly outperforms vanilla QMIX (blue line). HYPOLE +IQL (gray line) also achieves a non-trivial win rate (around 5%), whereas vanilla IQL (pink line) attains 0%. While both HYPOLE +QTRAN and vanilla QTRAN obtain 0% win rate, HYPOLE +QTRAN (brown line) eliminates more enemies than vanilla QTRAN (see Fig. 17i).

The remaining MessySMAC maps are shown in Fig. 14. In the 3m map Fig. 14a, HYPOLE +MARL substantially outperforms the vanilla MARL baselines across all cases. In the 2s3z map Fig. 14b, HYPOLE +QMIX (orange line) and HYPOLE +VDN (red line) substantially outperform vanilla QMIX (blue line) and vanilla VDN (green line), respectively.

To better illustrate the significance of HYPOLE, we report its success-rate gains across all SMAC maps in Fig. 10. HYPOLE +QMIX (blue line) outperforms vanilla QMIX on almost all maps, except 5m_vs_6m and 3s5z; however, on 3s5z, it still surpasses the baseline in the final training steps. On hard scenarios such as MMM2 Fig. 10g, bane_vs_bane Fig. 10h, and corridor Fig. 10i, HYPOLE +QMIX achieves substantial improvements, reaching up to a 90% gain in success rate on bane_vs_bane. Similarly, HYPOLE +VDN (orange line) improves performance in most cases, except on 2s3z, where the behavior is more unstable, while still achieving substantial gains on bane_vs_bane. Finally, HYPOLE +QTRAN (green line) outperforms vanilla QTRAN in nearly all scenarios except 5m_vs_6m, and achieves notable improvements on MMM, with up to a 50% gain Fig. 10f. In addition to CTDE methods, HYPOLE +IQL (red line) performs well on almost all maps, achieving about a 60% gain on 8m Fig. 10d.

We perform the same analysis for MessySMAC to further illustrate the significance of HYPOLE, and report its success-rate gains across all MessySMAC maps in Fig. 11. HYPOLE +QMIX (blue line) improves performance on most maps, especially

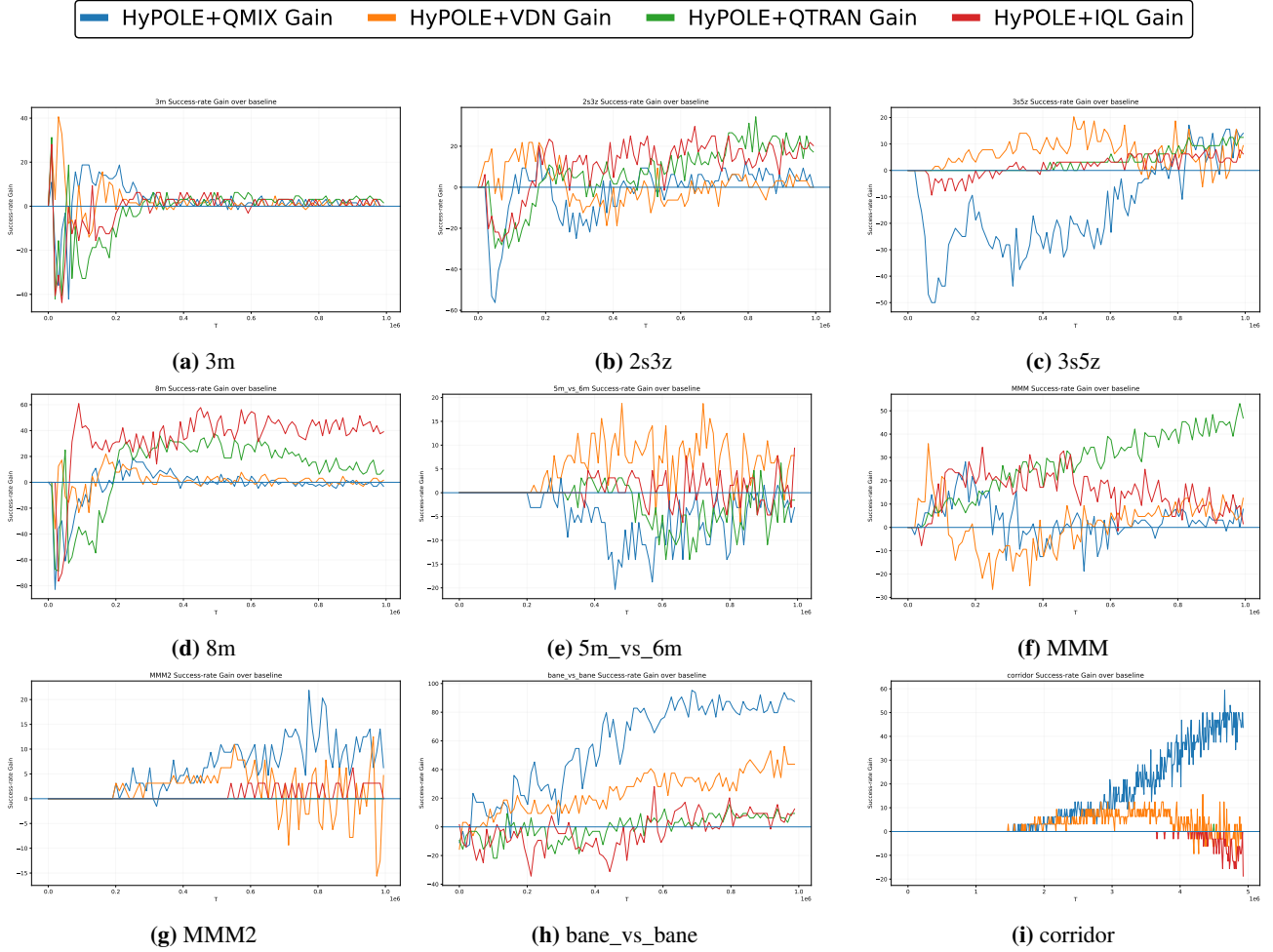


Figure 10. Median Gain in Percentage of HYPOLE compared to their respective baselines in SMAC.

on 3m Fig. 11a, where it achieves up to a 50% gain. HYPOLE +VDN (orange line) improves the win rate over vanilla VDN in all cases, and performs particularly well on the harder MMM map Fig. 11e, with up to a 30% gain. HYPOLE +QTRAN (green line) performs better in most cases, except on MMM Fig. 11e; however, it shows substantial improvement on 3m Fig. 11a, with up to a 50% gain. Similarly, HYPOLE +IQL (red line) performs better on most maps, especially on 3m Fig. 11a, with up to a $\approx 35\%$ gain, except on 8m Fig. 11d, where neither HYPOLE +IQL nor vanilla IQL achieves any wins.

We also report the number of dead enemies and dead allies across all SMAC maps in Figs. 16 and 17 and all MessySMAC maps in Figs. 18 and 19.

In Fig. 12, we report results on the WildFire scenarios, focusing on the number of steps required to save victims and extinguish fires. On the 5P_1F1V map (Figs. 12a to 12c), HYPOLE +MARL consistently outperforms the vanilla baselines, requiring fewer steps to complete the objectives. On the 5P_3F2V map (Figs. 12d to 12f), HYPOLE +MARL outperforms the vanilla baselines in most cases, except that QTRAN+ R_{Wild_2} and QTRAN+ R_{Wild_3} (purple line) achieve better performance than HYPOLE +QTRAN (brown line). On the 8P_3F2V map (Figs. 12g to 12i), HYPOLE +MARL outperforms the vanilla baselines in all cases; notably, HYPOLE +VDN (red line) and HYPOLE +IQL (gray line) perform substantially better than the vanilla variants across all baseline reward functions. On the 10P_3F2V map (Figs. 12j to 12l), HYPOLE +MARL outperforms the vanilla baselines in all cases; in particular, HYPOLE +VDN (red line) and HYPOLE +QMIX (orange line) significantly improve over the vanilla variants under all baseline reward functions.

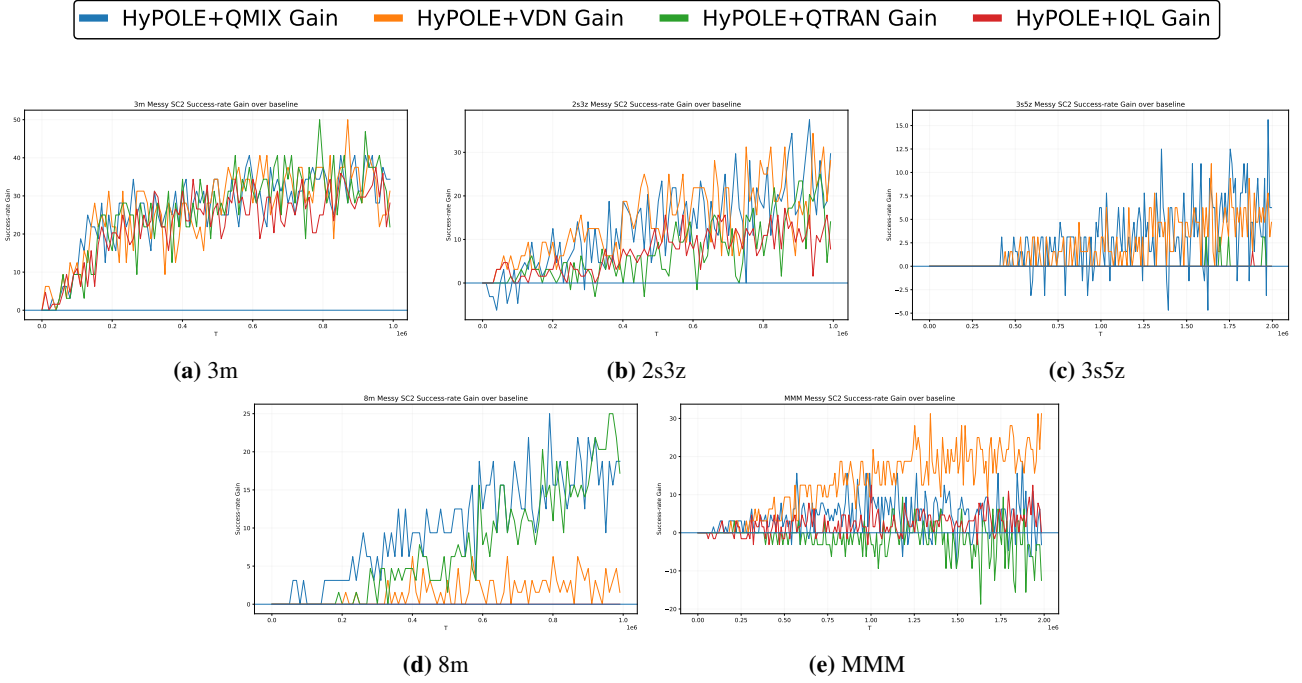


Figure 11. Median Gain in Percentage of HYPOLE compared to their respective baselines in MessySMAC.

H.2. $\forall\exists$ vs. $\forall\forall$ Specifications.

Since MMM2 includes a Medivac unit, we evaluate the φ_{medivac} specification using $\forall\exists$ -HyperLTL formulas. As shown in Fig. 15a, HYPOLE ($\forall\exists$)+QMIX (orange line) performs slightly better than HYPOLE ($\forall\forall$)+QMIX (blue line). We also provide same analysis for 10P_3F2V map in WildFire Fig. 15b and we saw HYPOLE ($\forall\exists$)+QTRAN (brown line) performs better than HYPOLE ($\forall\forall$)+QTRAN (purple line).

H.3. Visualization of tactics Learn by HYPOLE on SMAC

Fig. 20 shows snapshots after 1M training steps in SMAC. In Fig. 20a using φ_{defense} , the agents instead prioritize survival and retreat, moving away from enemies rather than committing to an engagement. Under φ_{medivac} Fig. 20b, the Medivac consistently moves toward the ally with the lowest health bar, indicating that the learned policy captures the intended support behavior. At the same time, the remaining agents exhibit focused firing by concentrating fire on a specific enemy target. Both Figs. 20a and 20b generated by VDN.

H.4. HYPOLE Computational Overhead

Next, we evaluate the computational overhead introduced by HYPOLE within the CTDE training pipeline. We run an experiment on MMM, with results reported in Table 3. In the $\forall\forall$ setting, HYPOLE incurs a negligible overhead of about 6% at 1000K environment steps. In contrast, in the $\forall\exists$ setting, the overhead increases to 62%, which is substantial. This experiment is run on an AMD EPYC 7742 64-core CPU and a single NVIDIA RTX A6000 GPU.

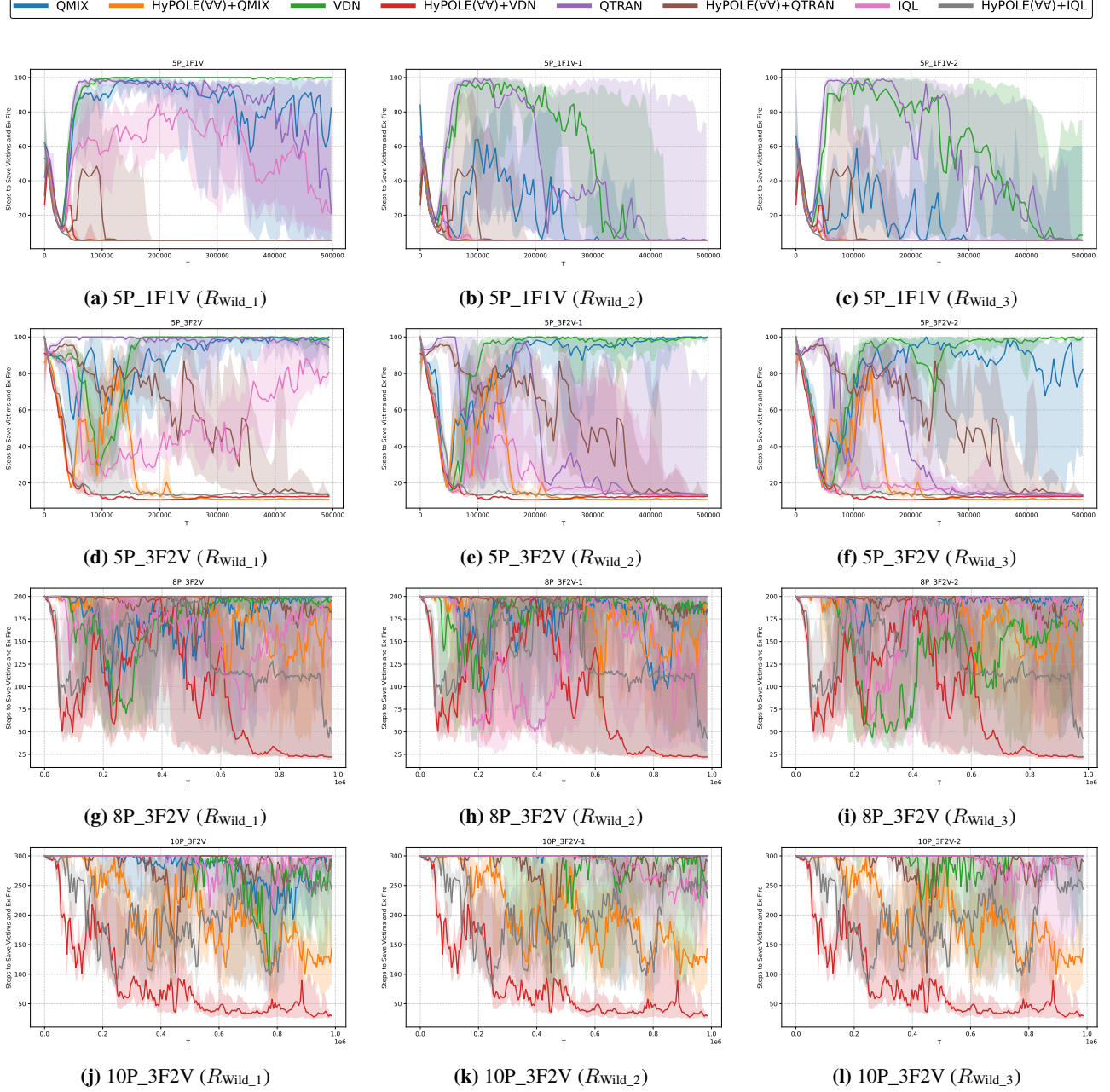


Figure 12. Learning curves on different WildFire Maps, showing median Steps to Extinguish Fire and Save Victim with 25-75% percentile.

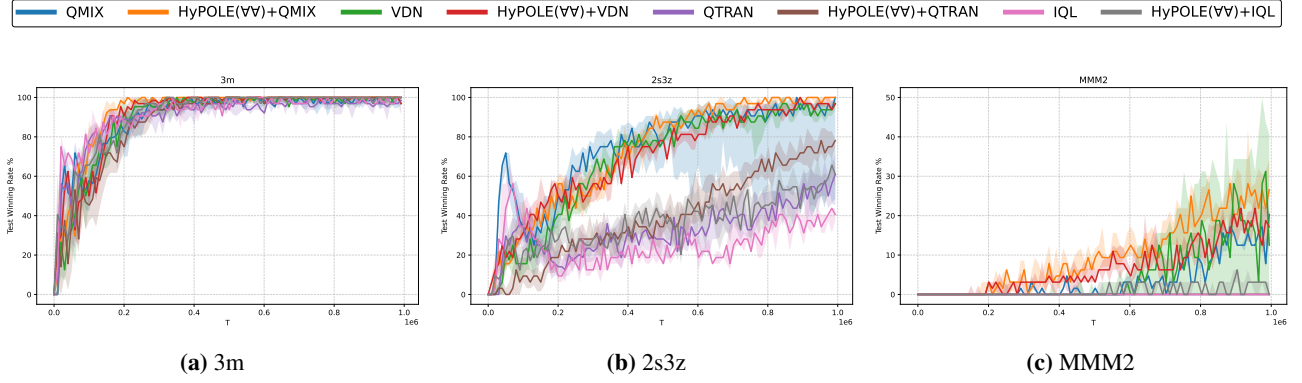


Figure 13. Learning curves on remaining SMAC maps, showing median test win rate with 25-75% percentile.

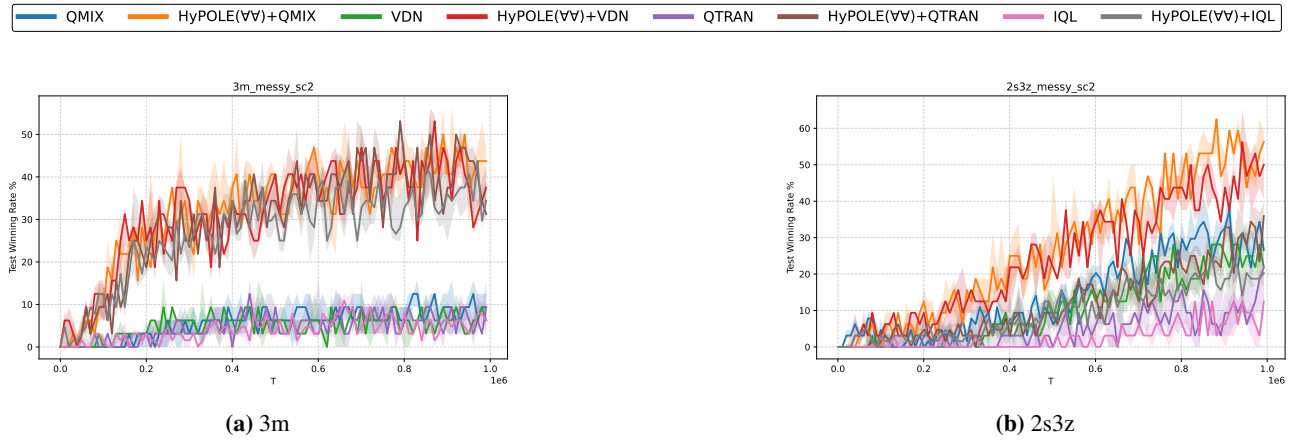


Figure 14. Learning curves on remaining MessySMAC maps, showing median test win rate with 25-75% percentile.

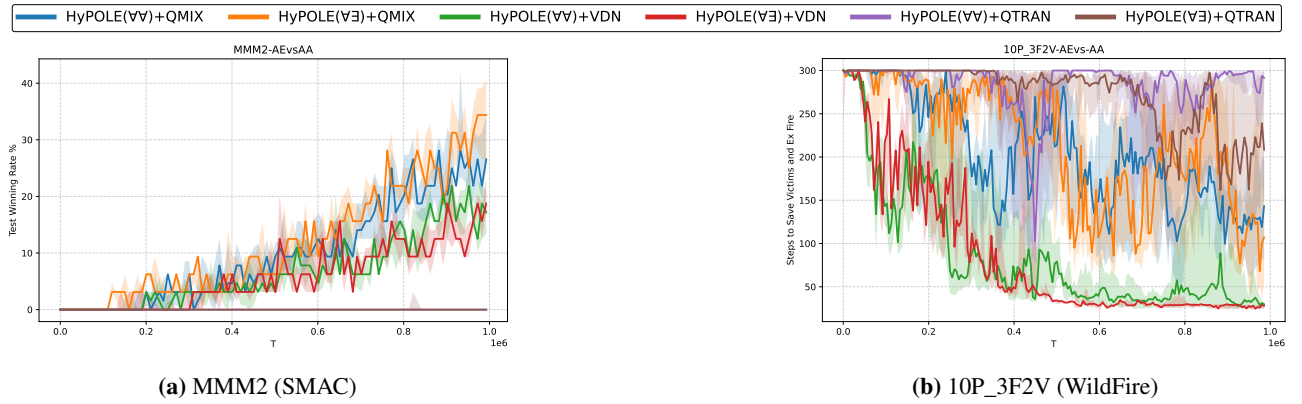


Figure 15. $\forall$ vs. $\forall\exists$ learning curves across SMAC, and WildFire, showing median test win rate with 25-75% percentiles.

Table 3. Mean execution time on the MMM map comparing MARL with and without HYPOLE. Results for the $\forall\exists$ formula are averaged only over VDN, QMIX, and QTRAN.

Method	200K Steps	400K Steps	600K Steps	800K Steps	1000K Steps
MARL	2202	4294	6305	8377	10385
HYPOLE ($\forall\forall$) + MARL	2211	4453	6637	8918	11100
HYPOLE ($\forall\exists$) + CTDE	3366	6821	10260	13659	16871

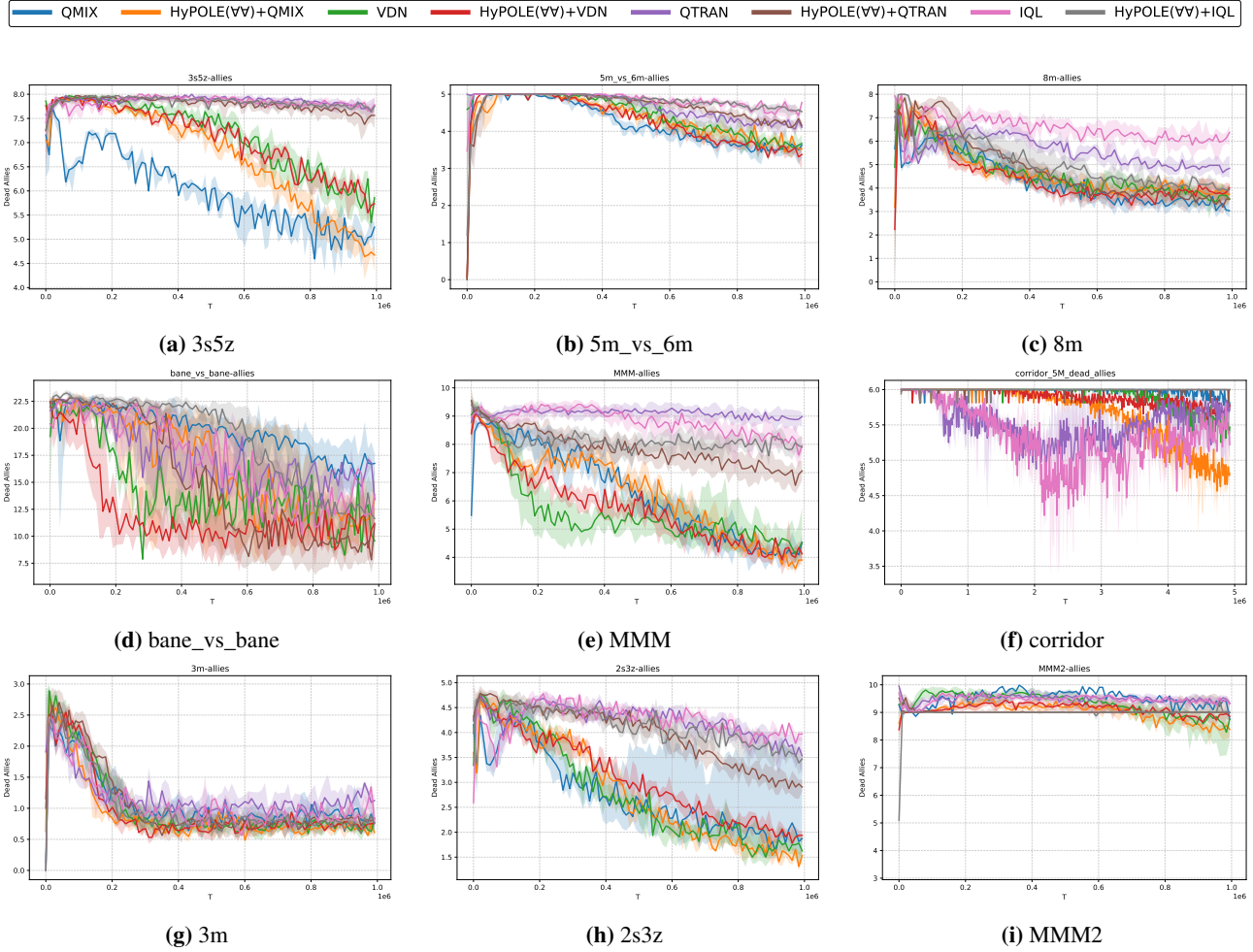


Figure 16. Learning curves on different SMAC combat maps, showing median dead ally agents with 25-75% percentile.

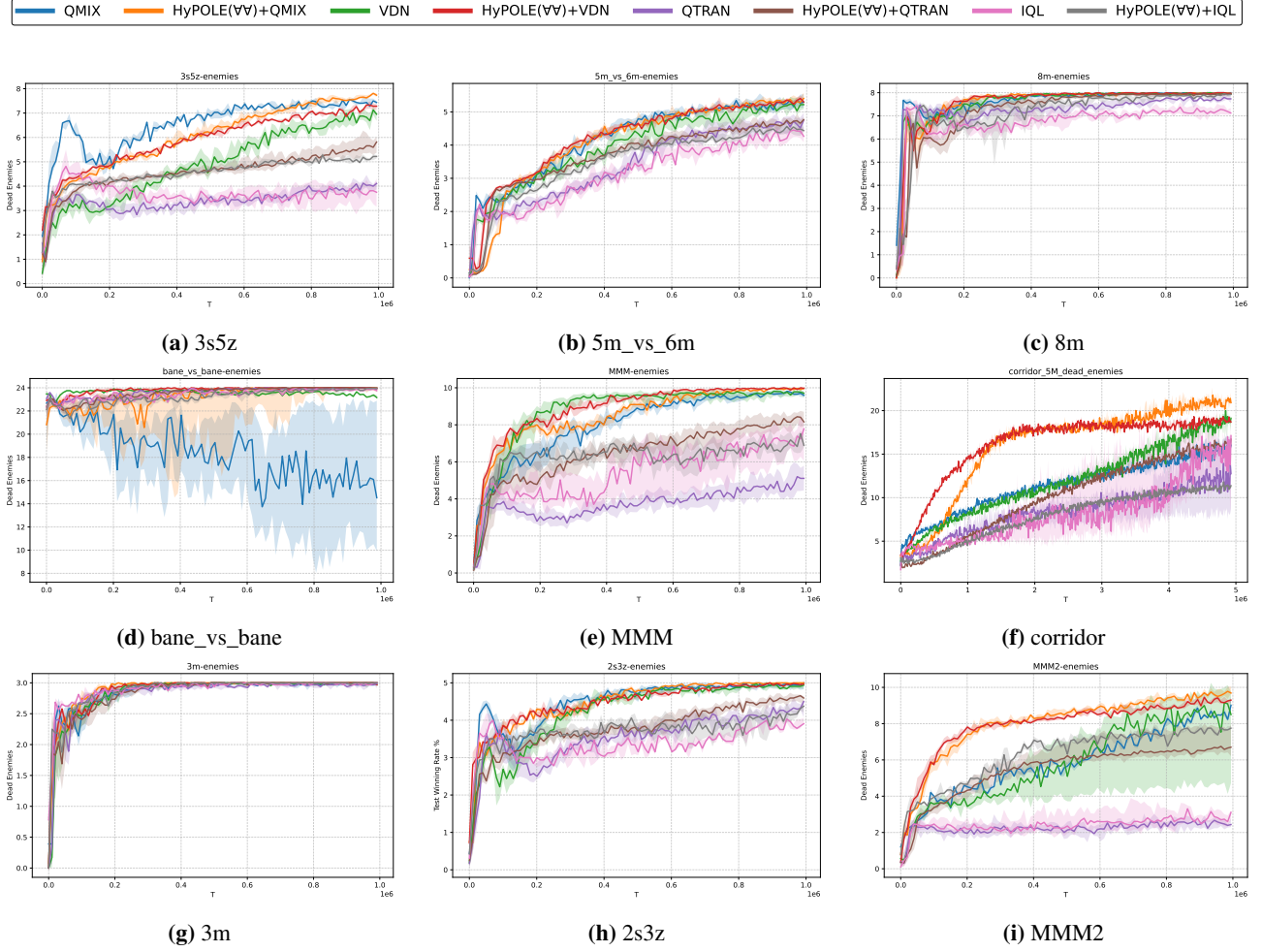


Figure 17. Learning curves on different SMAC combat maps, showing median dead enemy agents with 25-75% percentile.

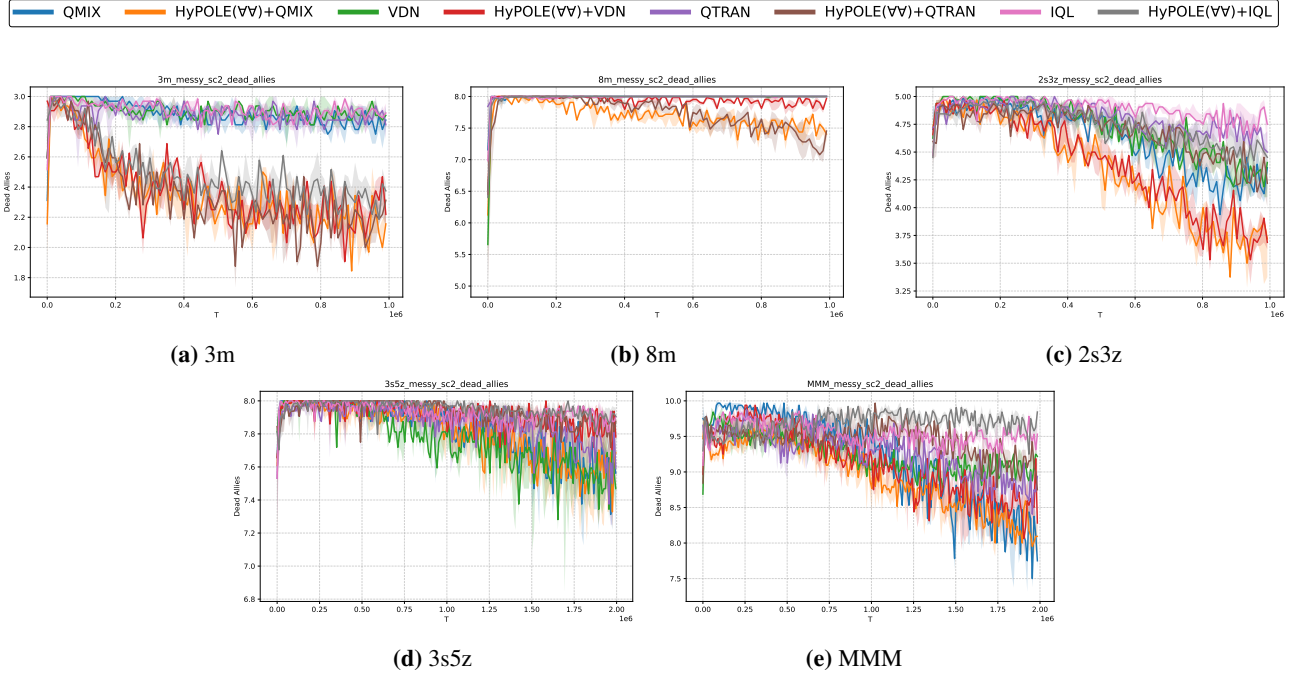


Figure 18. Learning curves on different MessySMAC combat maps, showing median dead ally agents with 25-75% percentile.

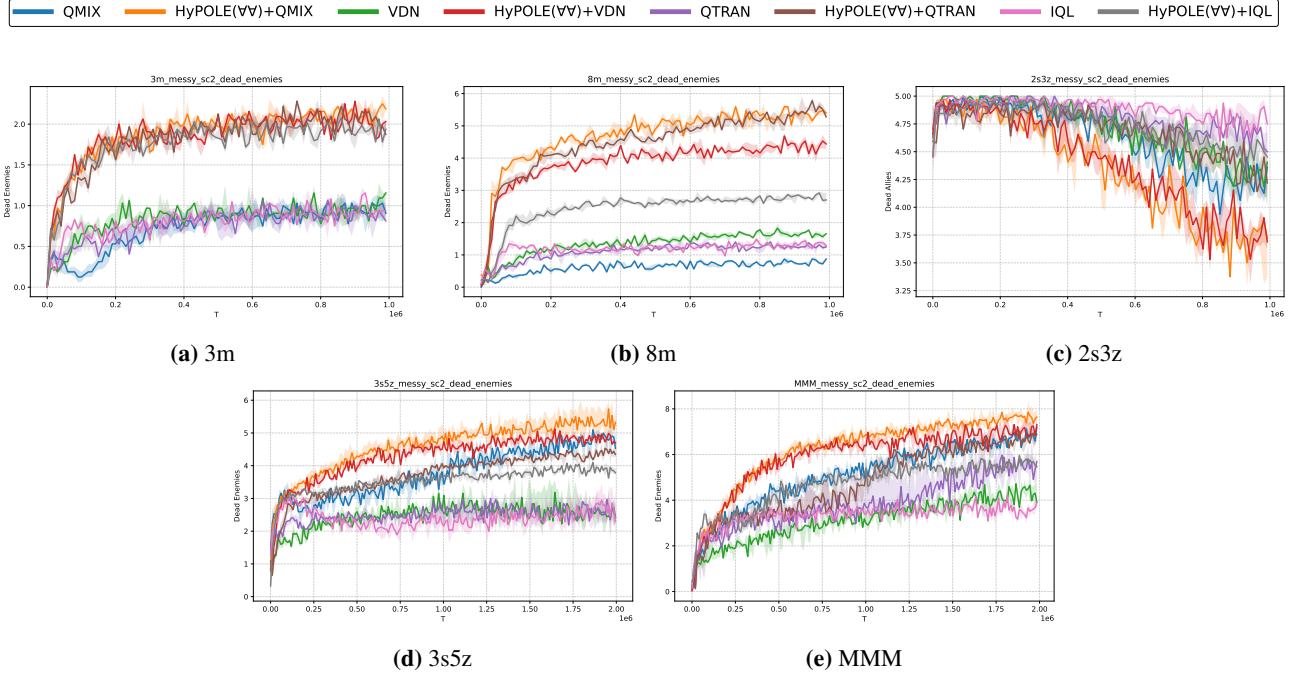


Figure 19. Learning curves on different MessySMAC showing median dead enemy agents with 25-75% percentile.



(a) φ_{defense} Tactic



(b) φ_{medic} Tactic

Figure 20. Snapshots of scenarios under HyperLTL-specified tactics.